

BỘ TÀI CHÍNH
TRƯỜNG ĐẠI HỌC TÀI CHÍNH – MARKETING
VIỆN CÔNG NGHỆ TÀI CHÍNH NGÂN HÀNG



BÁO CÁO MÔN HỌC

Tên đề tài:

Phát hiện giao dịch gian lận trong dữ liệu giao dịch tài chính

Bài toán: Phân lớp giao dịch bất thường trong luồng dữ liệu lớn.

Giảng viên hướng dẫn : ***Ths. Mai Thanh Thảo***

Lớp học phần : 2611101167901

Sinh viên thực hiện : Phan Quốc Tú

Ngày nộp : 18/04/2026

TP. HCM, THÁNG 04 NĂM 2026

BỘ TÀI CHÍNH
TRƯỜNG ĐẠI HỌC TÀI CHÍNH – MARKETING
VIỆN CÔNG NGHỆ TÀI CHÍNH NGÂN HÀNG



BÁO CÁO MÔN HỌC

Tên đề tài:

Phát hiện giao dịch gian lận trong dữ liệu giao dịch tài chính

Bài toán: Phân lớp giao dịch bất thường trong luồng dữ liệu lớn.

Giảng viên hướng dẫn : ***Ths. Mai Thanh Thảo***

Lớp học phần : 2611101167901

Sinh viên thực hiện : Phan Quốc Tú

Ngày nộp : 18/04/2026

TP. HCM, THÁNG 04 NĂM 2026

NHẬN XÉT – ĐÁNH GIÁ CỦA GIẢNG VIÊN CHẤM 1

[illegible]

Điểm số:.....

Điểm chữ:

Tp. HCM, ngày tháng năm 2026

Giảng viên

Ký tên (ghi rõ họ tên)

**NHẬN XÉT – ĐÁNH GIÁ CỦA
GIẢNG VIÊN CHẤM 2**

[illegible]

Điểm số:

Điểm chữ:.....

Tp. HCM, ngày tháng năm 2026

Giảng viên

Ký tên (ghi rõ họ tên)

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu cùng quý thầy cô Trường Đại học Tài chính – Marketing, những người đã không ngừng nỗ lực xây dựng một môi trường học tập năng động, hiện đại và chuyên nghiệp. Việc đưa học phần vào giảng dạy không chỉ giúp sinh viên nắm vững kiến thức lý thuyết mà còn tạo điều kiện để vận dụng vào thực tiễn thông qua các bài tiểu luận như thế này.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành đến Thạc sĩ **Mai Thanh Thảo** – giảng viên phụ trách học phần, người đã tận tình giảng dạy và hướng dẫn em trong suốt quá trình học tập và thực hiện bài luận. Sự tâm huyết, nghiêm túc và những góp ý chuyên môn quý báu từ cô đã giúp em có thêm động lực, định hướng đúng đắn cũng như hiểu sâu sắc hơn về nội dung môn học.

Mặc dù đã nỗ lực để hoàn thành bài luận với tinh thần nghiêm túc và trách nhiệm cao, nhưng do còn hạn chế về kiến thức và kinh nghiệm thực tiễn, bài viết chắc chắn không thể tránh khỏi những thiếu sót. Kính mong cô thông cảm, đồng thời rất mong nhận được những góp ý chân thành để em có thể tiếp tục cải thiện và hoàn thiện hơn trong quá trình học tập sau này.

Một lần nữa, em xin gửi lời cảm ơn chân thành và kính chúc cô thật nhiều sức khỏe, luôn thành công và tiếp tục truyền cảm hứng đến các thế hệ sinh viên.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 04 năm 2026

Người viết báo cáo

Phan Quốc Tú

MỤC LỤC

LỜI CẢM ƠN.....	i
MỤC LỤC	ii
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU	vii
DANH MỤC TỪ VIẾT TẮT	viii
TÓM TẮT.....	1
CHƯƠNG 1. GIỚI THIỆU VÀ CƠ SỞ LÝ THUYẾT	2
1.1. Bối cảnh bài toán	2
1.2. Cơ sở lý thuyết của đề tài	3
1.2.1. Khái niệm phát hiện gian lận trong thanh toán trực tuyến	3
1.2.2. Dữ liệu mất cân bằng và hệ quả đối với đánh giá mô hình	4
1.2.3. Vai trò của kỹ thuật đặc trưng trong fraud detection	4
1.2.4. Học máy có giám sát và lựa chọn mô hình	5
1.2.5. Ngưỡng quyết định và bài toán vận hành.....	5
1.2.6. Cơ sở lý thuyết cho thiết kế hệ thống dữ liệu lớn	6
1.3. Mục tiêu nghiên cứu	6
1.4. Đối tượng, phạm vi và dữ liệu nghiên cứu	7
1.5. Phương pháp nghiên cứu	7
1.6. Ý nghĩa khoa học và thực tiễn	8
CHƯƠNG 2. PHƯƠNG PHÁP LUẬN VÀ TIỀN XỬ LÝ DỮ LIỆU	9
2.1. Khám phá dữ liệu (EDA).....	9
2.1.1. Biện luận về chiến lược tối ưu RAM và xử lý theo chunk.....	9
2.1.2. Mô tả nguồn dữ liệu	9

2.1.3. Kiểm tra cấu trúc dữ liệu và chuẩn hóa kiểu dữ liệu cho EDA.....	10
2.1.4. Khám phá dữ liệu thiếu	11
2.1.5. Khám phá phân bố lớp mục tiêu.....	12
2.1.6. Khám phá phân bố amount và ngoại lệ	13
2.1.7. Tương quan và phân tích phụ thuộc giữa các biến số	13
2.1.8. Phân tích theo thời gian	15
2.2. Quy trình tiền xử lý dữ liệu	16
2.2.1. Tích hợp đa nguồn dữ liệu (Merge/Join).....	16
2.2.2. Làm sạch dữ liệu: missing values, dữ liệu nhiễu và trùng lặp	17
2.2.3. Biến đổi dữ liệu: chuẩn hóa và mã hóa	18
2.2.4. Sơ đồ quan hệ ERD	19
2.3. Kỹ thuật đặc trưng	20
2.3.1. Đặc trưng thời gian.....	20
2.3.2. Đặc trưng tương tác	20
2.3.3. Đặc trưng tích lũy theo khách hàng.....	20
2.4. Kết luận chương 2	21
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG DỮ LIỆU LỚN	22
3.1. Kiến trúc hệ thống	22
3.1.1. Biện luận lựa chọn kiến trúc.....	22
3.1.2. Các lớp thành phần của hệ thống	23
3.1.3. Luồng dữ liệu batch và speed path	24
3.2. Thiết kế lưu trữ (Schema Design)	25
3.2.1. Nguyên tắc thiết kế schema.....	25
3.2.2. Cấu trúc document mẫu.....	26
3.2.3. Chỉ mục và tối ưu truy vấn	27

3.3. Tính toán hạ tầng lưu trữ	28
3.3.1. Cơ sở tính toán.....	28
3.3.2. Kết quả ước tính	28
3.3.3. Pipeline nạp dữ liệu và thử nghiệm trên Google Colab	29
3.3.4. Biện luận lựa chọn MongoDB Atlas thay vì SQL.....	29
3.4. Dashboard giám sát gian lận và công bố qua ngrok.....	30
3.4.1. Mục tiêu thiết kế dashboard	30
3.4.3. Nguồn dữ liệu cấp cho dashboard và logic truy vấn	31
3.4.4. Kiến trúc giao diện và nguyên tắc thiết kế trực quan.....	33
3.4.5. Mô tả chi tiết các phân hệ giao diện.....	33
3.4.7. Cơ chế mô phỏng gần thời gian thực và đồng bộ dữ liệu	36
3.4.8. Công bố dashboard qua ngrok.....	37
3.5. Kết luận chương 3	38
CHƯƠNG 4. THỰC NGHIỆM KHAI PHÁ VÀ KẾT QUẢ	39
4.1. Xây dựng mô hình	39
4.1.1. Dữ liệu đầu vào cho mô hình	39
4.1.2. Các mô hình được thực nghiệm	39
4.1.3. Tinh chỉnh siêu tham số bằng GridSearchCV	40
4.1.4. Kết quả so sánh mô hình trên validation	40
4.2. Đánh giá kỹ thuật.....	41
4.2.1. Vì sao không dùng Accuracy làm chỉ số chính	41
4.2.2. Đánh giá tại ngưỡng mặc định 0,5	42
4.2.3. Threshold tuning và lựa chọn ngưỡng vận hành	42
4.3. Phân tích tác động kinh tế và khả năng triển khai.....	44
4.3.1. Chuyển từ đánh giá kỹ thuật sang đánh giá nghiệp vụ.....	44

4.3.2. Giả định chi phí – lợi ích	44
4.3.3. Giá trị kinh tế ròng của mô hình.....	45
4.3.4. Đánh giá threshold dưới góc nhìn vận hành.....	46
4.3.5. Phân tầng rủi ro để phục vụ dashboard	46
4.3.6. Cập nhật thiết kế về hiệu quả kinh tế	47
4.3.7. Lưu ý về business insight từ feature importance.....	51
4.4. Kết luận chương 4	51
CHƯƠNG 5. KẾT LUẬN VÀ KIẾN NGHỊ	53
5.1. Kết luận.....	53
5.2. Kiến nghị thực tế	53
5.3. Hạn chế của đề tài.....	54
5.4. Định hướng phát triển.....	54
TÀI LIỆU THAM KHẢO	55

DANH MỤC HÌNH ẢNH

Hình 2. 1: Ma trận dữ liệu thiếu trên mẫu giao dịch (minh họa cho các cột thiếu chính) ..	12
Hình 2. 2: Phân bố lớp mục tiêu của bài toán fraud detection	12
Hình 2. 3: Heatmap tương quan giữa các biến số chính trên mẫu EDA	14
Hình 2. 4: Phân bố giao dịch và tỷ lệ fraud theo giờ	15
Hình 2. 5: Phân bố giao dịch và tỷ lệ fraud theo ngày trong tuần	15
Hình 2. 6: Sơ đồ dữ liệu ERD	19
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG DỮ LIỆU LỚN	
Hình 3. 1: Kiến trúc logic của hệ thống phát hiện gian lận trên dữ liệu giao dịch lớn	23
Hình 3. 2: Sơ đồ bố cục logic của dashboard giám sát gian lận	31
Hình 3. 3: Ảnh chụp kết quả thực nghiệm: tab Duyệt Giao dịch Rủi ro trên dashboard ...	36
Hình 3. 4: Chu trình cập nhật gần thời gian thực của dashboard	37
CHƯƠNG 4: THỰC NGHIỆM KHAI PHÁ VÀ KẾT QUẢ	
Hình 4. 1: So sánh ba mô hình trên tập validation	41
Hình 4. 2: So sánh hiệu quả giữa threshold mặc định và threshold vận hành	43
Hình 4. 3: Cấu phần giá trị kinh tế ròng của mô hình tại threshold = 0,17	45
Hình 4. 4: Phân bố giao dịch theo ba mức rủi ro trên dashboard	47
Hình 4. 5: Dashboard sau khi bổ sung KPI “Giá trị kinh tế” và liên kết với tab duyệt giao dịch rủi ro	49
Hình 4. 6: Tab “Giá trị kinh tế” hiển thị KPI	50
Hình 4. 7: Biểu đồ xu hướng giá trị kinh tế tích lũy theo thời gian dựa trên collection business_value_snapshots	50

DANH MỤC BẢNG BIỂU

Bảng 2. 1: Các trường thông tin cốt lõi của bộ dữ liệu	10
Bảng 2. 2: Cấu trúc dữ liệu sau chuẩn hóa trên mẫu EDA.....	11
Bảng 2. 3: Tóm tắt dữ liệu thiếu trên transactions_data.csv	11
Bảng 2. 4: Phân bố lớp fraud_label	12
Bảng 2. 5: Kết quả phát hiện outliers trên biến amount	13
Bảng 2. 6: Kết quả sau bước tích hợp dữ liệu	16
Bảng 2. 7: Các quyết định chính trong bước làm sạch.....	17
Bảng 2. 8: Các bước biến đổi phục vụ mô hình	18
Bảng 2. 9: Các đặc trưng được tạo thêm ở bước feature engineering	21
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG DỮ LIỆU LỚN	
Bảng 3. 1: Các collection chính của hệ thống MongoDB	26
Bảng 3. 2: Ví dụ các trường cốt lõi trong từng collection.....	27
Bảng 3. 3: Các tham số cơ sở để ước tính lưu trữ	28
Bảng 3. 4: Kích bản dung lượng lưu trữ trên MongoDB Atlas.....	28
Bảng 3. 5: Mapping giữa thành phần giao diện và collection MongoDB.....	32
CHƯƠNG 4: THỰC NGHIỆM KHAI PHÁ VÀ KẾT QUẢ	
Bảng 4. 1: Tổng quan dữ liệu đánh giá mô hình	39
Bảng 4. 2: So sánh kết quả huấn luyện trên tập validation	40
Bảng 4. 3: Kết quả XGBoost trên tập test tại threshold = 0,5	42
Bảng 4. 4: Kết quả cuối của XGBoost tại threshold = 0,17	43
Bảng 4. 5: Bộ giả định chi phí – lợi ích trong phân tích kinh tế	45
Bảng 4. 6: Ước tính tác động kinh tế tại threshold = 0,17	45
Bảng 4. 7: Chỉ số vận hành của threshold = 0,17	46

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Thuật ngữ	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
AUC	Area Under the Curve	Diện tích dưới đường cong ROC
CSV	Comma-Separated Values	Tập dữ liệu dạng phân tách bởi dấu phẩy
EDA	Exploratory Data Analysis	Phân tích khám phá dữ liệu
FN	False Negative	Dự đoán âm tính sai
FP	False Positive	Dự đoán dương tính sai
KPI	Key Performance Indicator	Chỉ số hiệu suất trọng yếu
ML	Machine Learning	Học máy
NoSQL	Not Only SQL	Hệ quản trị dữ liệu phi quan hệ
RF	Replication Factor	Hệ số nhân bản dữ liệu
ROC	Receiver Operating Characteristic	Đường cong đặc trưng hoạt động của bộ phân loại
SMOTE	Synthetic Minority Over-sampling Technique	Kỹ thuật tăng mẫu lớp thiểu số
TN	True Negative	Dự đoán âm tính đúng
TP	True Positive	Dự đoán dương tính đúng

TÓM TẮT

Đề tài tập trung xây dựng một quy trình phát hiện gian lận trong thanh toán trực tuyến trên bộ dữ liệu giao dịch tài chính lớn từ Kaggle. Xuất phát từ đặc trưng dữ liệu có quy mô hàng chục triệu bản ghi, nhiều nguồn khác nhau và tỷ lệ fraud cực thấp, nghiên cứu lựa chọn cách tiếp cận kết hợp giữa tiền xử lý dữ liệu lớn, học máy có giám sát và thiết kế hệ thống hỗ trợ giám sát gần thời gian thực. Trong Chương 2, dữ liệu giao dịch và dữ liệu nhãn được tích hợp bằng phép inner join, sau đó được làm sạch, chuẩn hóa kiểu dữ liệu, xử lý missing values, gán cờ outliers và tạo các đặc trưng thời gian, tương tác và tích lũy theo khách hàng. Chương 3 trình bày kiến trúc dữ liệu lớn theo hướng kết hợp batch path, speed path và serving layer; dữ liệu được tổ chức trên MongoDB Atlas với các collection phục vụ truy vấn lịch sử, dự đoán, cảnh báo và dashboard. Chương 4 thực nghiệm ba mô hình Logistic Regression, Random Forest và XGBoost; kết quả cho thấy XGBoost là mô hình phù hợp nhất. Sau threshold tuning, mô hình đạt Recall = 0,932108, ROC-AUC = 0,986074 tại ngưỡng 0,17; đồng thời mô hình tạo giá trị kinh tế ròng dương ước tính 255.815 đơn vị giá trị theo bộ giả định chi phí của đề tài. Kết quả cho thấy hệ thống phù hợp nhất với vai trò cảnh báo và hỗ trợ rà soát.

CHƯƠNG 1. GIỚI THIỆU VÀ CƠ SỞ LÝ THUYẾT

1.1. Bối cảnh bài toán

Trong hơn một thập niên gần đây, thanh toán điện tử, ngân hàng số và thương mại điện tử đã làm thay đổi sâu sắc cách thức giao dịch tài chính được thực hiện trong nền kinh tế. Sự gia tăng của số lượng giao dịch trực tuyến giúp doanh nghiệp mở rộng quy mô dịch vụ và cải thiện trải nghiệm khách hàng, nhưng đồng thời cũng tạo ra bề mặt tấn công lớn hơn cho các hành vi gian lận. Fraud trong thanh toán không chỉ gây tổn thất tài chính trực tiếp cho tổ chức phát hành thẻ, ngân hàng hoặc đơn vị chấp nhận thanh toán, mà còn kéo theo chi phí điều tra, hoàn tiền, xử lý khiếu nại, gián đoạn vận hành và suy giảm niềm tin của khách hàng. Chính vì vậy, phát hiện gian lận giao dịch đã trở thành một bài toán có ý nghĩa thực tiễn rất cao trong cả nghiên cứu và vận hành hệ thống tài chính hiện đại (Caixabank Tech, 2024), (Güven, 2016).

Về mặt bản chất, fraud detection trong thanh toán trực tuyến là bài toán phân lớp nhị phân có giám sát: mỗi giao dịch được gán vào một trong hai lớp là giao dịch bình thường hoặc giao dịch gian lận. Tuy nhiên, đây không phải là một bài toán phân lớp thông thường. Giao dịch gian lận chỉ chiếm một tỷ lệ rất nhỏ so với tổng số giao dịch; vì vậy, nếu chỉ tối ưu độ chính xác tổng thể, mô hình có thể dễ dàng thiên lệch về lớp đa số và vẫn cho ra Accuracy cao nhưng gần như vô dụng về mặt nghiệp vụ. Trong bối cảnh này, các chỉ số như Recall, Precision, F1-score và ROC-AUC có ý nghĩa quan trọng hơn vì chúng phản ánh khả năng phát hiện đúng fraud và mức đánh đổi giữa cảnh báo đúng và cảnh báo sai (Garcia, 2009), (N. V. Chawla, 2002).

Đặc trưng của bài toán còn nằm ở chỗ hành vi gian lận thường thay đổi theo thời gian, phụ thuộc vào bối cảnh giao dịch, cách thức sử dụng thẻ, nhóm merchant và thói quen của từng khách hàng. Một tín hiệu có thể bình thường với nhóm khách hàng này nhưng lại bất thường với nhóm khác. Do đó, việc phát hiện gian lận không thể chỉ dựa trên một vài biến tĩnh, mà cần khai thác quan hệ giữa thời gian, số tiền, ngữ cảnh giao dịch và lịch sử hành vi. Từ góc nhìn học thuật, điều này đòi hỏi quy trình nghiên cứu phải bao trùm từ tiền xử

lý dữ liệu, kỹ thuật đặc trưng, lựa chọn mô hình, tinh chỉnh ngưỡng cảnh báo đến tổ chức đầu ra phục vụ giám sát.

Từ góc nhìn hệ thống, bài toán của đề tài còn gắn với xử lý dữ liệu lớn. Khi quy mô dữ liệu lên đến hàng triệu hoặc hàng chục triệu bản ghi, việc nạp toàn bộ dữ liệu vào bộ nhớ và xử lý theo cách tuần tự thông thường sẽ dẫn đến áp lực tài nguyên lớn, đặc biệt trên môi trường nghiên cứu như Google Colab. Điều đó khiến đề tài không chỉ là bài toán mô hình hóa, mà còn là bài toán thiết kế pipeline dữ liệu đủ ổn định để làm sạch, tích hợp, tạo đặc trưng, lưu trữ và hiển thị kết quả một cách nhất quán (Apache Spark, 2025), (MongoDB, 2025)].

1.2. Cơ sở lý thuyết của đề tài

1.2.1. Khái niệm phát hiện gian lận trong thanh toán trực tuyến

Phát hiện gian lận trong thanh toán trực tuyến là quá trình sử dụng dữ liệu giao dịch và các kỹ thuật phân tích để nhận diện các giao dịch có dấu hiệu bất thường hoặc có khả năng là gian lận. Trong môi trường vận hành thực tế, đầu ra của hệ thống thường không chỉ là nhãn nhị phân 0/1 mà còn là một điểm rủi ro (fraud score) để bộ phận nghiệp vụ có thể thiết lập ngưỡng hành động phù hợp. Mục tiêu cuối cùng không phải là đạt độ chính xác thống kê cao nhất, mà là giảm thiểu thiệt hại tài chính do fraud gây ra với chi phí kiểm tra và tác động đến khách hàng ở mức chấp nhận được.

Hai loại sai số trong fraud detection có ý nghĩa nghiệp vụ khác nhau. False Negative là trường hợp hệ thống bỏ sót một giao dịch gian lận; đây thường là sai số có chi phí rất cao vì dẫn đến tổn thất tài chính trực tiếp. False Positive là trường hợp hệ thống gán cờ nhầm một giao dịch hợp lệ; sai số này gây chi phí rà soát và có thể làm phiền khách hàng, nhưng trong nhiều bối cảnh vẫn được chấp nhận hơn so với bỏ sót fraud. Vì vậy, hệ thống phát hiện gian lận thường ưu tiên giảm FN, đồng nghĩa với việc phải cân nhắc hạ ngưỡng quyết định và chấp nhận tăng FP ở mức có thể kiểm soát.

Một mô hình fraud detection tốt cần thỏa mãn ít nhất ba yêu cầu. Thứ nhất, nó phải phát hiện được phần lớn fraud thật, tức Recall phải đủ cao. Thứ hai, nó phải tạo ra đầu ra có thể diễn giải hoặc ít nhất có thể tổ chức thành tin hiệu rủi ro phục vụ dashboard. Thứ ba,

nó phải đủ nhanh để tích hợp vào quy trình xử lý batch hoặc micro-batch, thay vì chỉ hữu ích trong nghiên cứu ngoại tuyến.

1.2.2. Dữ liệu mất cân bằng và hệ quả đối với đánh giá mô hình

Dữ liệu mất cân bằng là hiện tượng lớp mục tiêu quan tâm chiếm tỷ trọng rất nhỏ so với lớp còn lại. Trong fraud detection, đây là đặc điểm phổ biến: phần lớn giao dịch là hợp lệ, trong khi giao dịch gian lận chỉ là một tỷ lệ rất nhỏ. Trên bộ dữ liệu của đề tài, tỷ lệ fraud chỉ khoảng 0,1495%, nghĩa là cứ khoảng 1.000 giao dịch mới có chưa đến 2 giao dịch fraud. Với đặc điểm như vậy, nếu mô hình luôn dự đoán “không fraud”, Accuracy vẫn gần như tuyệt đối. Điều này cho thấy Accuracy không phản ánh đúng giá trị sử dụng của mô hình trong bài toán phát hiện gian lận.

Khi dữ liệu bị mất cân bằng mạnh, việc đánh giá mô hình cần chuyển từ câu hỏi “mô hình đúng bao nhiêu phần trăm tổng thể” sang câu hỏi “mô hình phát hiện được bao nhiêu fraud và phải trả giá bằng bao nhiêu cảnh báo sai”. Recall đo tỷ lệ fraud được phát hiện đúng trên tổng số fraud thật; Precision đo tỷ lệ fraud thật trong số các giao dịch bị gán cờ; F1-score cân bằng giữa hai chỉ số này; còn ROC-AUC đo năng lực phân biệt hai lớp trên toàn bộ dải ngưỡng (Garcia, 2009), (N. V. Chawla, 2002).

Để cải thiện khả năng học của mô hình trên lớp thiểu số, nhiều chiến lược đã được sử dụng trong nghiên cứu, như điều chỉnh trọng số lớp, lấy mẫu lại dữ liệu hoặc tạo mẫu tổng hợp bằng SMOTE. Tuy nhiên, các kỹ thuật này cần được áp dụng đúng vị trí trong pipeline, tránh làm rò rỉ thông tin giữa train và validation/test. Trong đề tài này, SMOTE chỉ được áp dụng cho tập huấn luyện sau khi chia dữ liệu, nhằm cải thiện khả năng học của mô hình mà không làm sai lệch đánh giá ở tập kiểm tra.

1.2.3. Vai trò của kỹ thuật đặc trưng trong fraud detection

Trong các bài toán giao dịch tài chính, dữ liệu thô thường chỉ phản ánh trực tiếp sự kiện giao dịch tại một thời điểm, chẳng hạn số tiền, giờ giao dịch, phương thức sử dụng thẻ hay thông tin merchant. Nếu sử dụng trực tiếp các biến này, mô hình có thể nhận được một phần tín hiệu nhưng chưa đủ để mô tả hành vi. Kỹ thuật đặc trưng vì vậy đóng vai trò biến dữ liệu thô thành các tín hiệu có ý nghĩa hơn về mặt kinh tế và hành vi.

Nhóm đặc trưng thời gian giúp mô hình nhận diện các chu kỳ có nguy cơ cao, ví dụ giao dịch ban đêm, cuối tuần hoặc các khung giờ bất thường. Nhóm đặc trưng tương tác kết

hợp nhiều yếu tố, chẳng hạn $\text{amount} \times \text{is_night}$, để nêu bật trường hợp giao dịch lớn xuất hiện vào giờ nhạy cảm. Nhóm đặc trưng tích lũy theo khách hàng hoặc thẻ cho phép mô hình so sánh giao dịch hiện tại với hành vi lịch sử của chính đối tượng đó, ví dụ số tiền hiện tại lệch bao nhiêu so với mức trung bình trước đây. Trong fraud detection, các đặc trưng loại này thường hữu ích hơn nhiều so với việc đưa trực tiếp mã định danh vào mô hình.

Từ góc nhìn nghiệp vụ, feature engineering còn là cầu nối giữa khai phá dữ liệu và quyết định quản trị. Nếu đặc trưng được xây dựng bám sát logic giao dịch, kết quả mô hình sẽ dễ được chuyển hóa thành dashboard, quy tắc cảnh báo và phân tầng rủi ro hơn.

1.2.4. Học máy có giám sát và lựa chọn mô hình

Học máy có giám sát được lựa chọn trong đề tài vì dữ liệu đã có nhãn `fraud_label` sau bước tích hợp. Ba mô hình được thực nghiệm gồm Logistic Regression, Random Forest và XGBoost. Logistic Regression đóng vai trò mô hình baseline nhờ tính đơn giản, dễ triển khai và dễ diễn giải. Random Forest đại diện cho nhóm mô hình ensemble bagging, có khả năng khai thác quan hệ phi tuyến và giảm phương sai. XGBoost đại diện cho boosting, nổi bật ở khả năng học các quan hệ phức tạp và thường cho hiệu quả cao trên dữ liệu dạng bảng (Guestrin, 2016).

Bên cạnh lựa chọn thuật toán, một điểm quan trọng là tiêu chí tối ưu trong giai đoạn GridSearchCV. Do bài toán nhấn mạnh khả năng phát hiện fraud, đề tài ưu tiên Recall làm tiêu chí chính khi lựa chọn siêu tham số. Cách làm này nhất quán với yêu cầu nghiệp vụ: chấp nhận cảnh báo rộng hơn để giảm bỏ sót fraud. Tuy nhiên, để mô hình có thể sử dụng được trong thực tế, việc tối ưu Recall sau đó vẫn cần được kiểm chứng lại bằng Precision, F1-score, ROC-AUC và phân tích threshold.

1.2.5. Ngưỡng quyết định và bài toán vận hành

Nhiều bài toán phân lớp dừng ở nhãn dự đoán 0/1 tại ngưỡng mặc định 0,5. Tuy nhiên, với fraud detection, ngưỡng 0,5 chưa chắc là lựa chọn tốt nhất. Nếu ngưỡng quá cao, Precision có thể tăng nhưng Recall giảm và hệ thống bỏ sót nhiều fraud. Nếu ngưỡng quá thấp, Recall tăng nhưng số lượng FP cũng tăng mạnh, dẫn đến khối lượng rà soát lớn. Vì vậy, threshold tuning là bước bắt buộc để chuyển một mô hình kỹ thuật thành một công cụ vận hành.

Một hệ thống phát hiện gian lận phù hợp trong thực tế thường có ba lớp quyết định. Lớp thứ nhất là fraud score liên tục do mô hình sinh ra. Lớp thứ hai là predicted_label tại ngưỡng vận hành. Lớp thứ ba là phân tầng rủi ro, ví dụ high, medium và low, để dashboard dễ hiển thị và bộ phận nghiệp vụ dễ ưu tiên. Chính cơ chế nhiều lớp này giúp nối mô hình học máy với hệ thống lưu trữ và giám sát.

1.2.6. Cơ sở lý thuyết cho thiết kế hệ thống dữ liệu lớn

Ở quy mô dữ liệu lớn, một pipeline fraud detection cần vừa giải quyết lớp mô hình hóa vừa giải quyết lớp hạ tầng dữ liệu. Dữ liệu lịch sử đã làm sạch và tạo đặc trưng cần được lưu trữ trong một kho trung tâm để phục vụ truy vấn, kiểm tra, đối chiếu và hiển thị dashboard. Dữ liệu dự đoán và cảnh báo cần được ghi riêng để theo dõi lịch sử mô hình và hỗ trợ audit. Điều này khiến một hệ quản trị dữ liệu linh hoạt như MongoDB Atlas trở thành lựa chọn phù hợp cho đề tài, đặc biệt khi dữ liệu đầu ra có dạng document và có thể mở rộng thêm các trường metadata trong tương lai (MongoDB, 2025).

Về kiến trúc, đề tài vận dụng tư duy của Lambda Architecture ở mức logic: dữ liệu lịch sử đi theo batch path, dữ liệu mới đi theo speed path, và các kết quả hội tụ ở serving layer để cung cấp cho dashboard. Đây là cách thiết kế phù hợp với yêu cầu của hướng dẫn báo cáo: luồng đi phải thể hiện rõ từ vấn đề kinh tế, thu thập và làm sạch dữ liệu, hạ tầng lưu trữ, khai phá dữ liệu cho đến giải pháp kinh doanh. Trong bối cảnh đồ án, kiến trúc được đơn giản hóa để có thể triển khai trên Colab – MongoDB – Streamlit – ngrok nhưng vẫn giữ logic của một hệ thống dữ liệu lớn (Apache Spark, 2025), (MongoDB, 2025).

1.3. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng một quy trình phát hiện gian lận trong thanh toán trực tuyến trên dữ liệu giao dịch lớn, trong đó dữ liệu được tiền xử lý nhất quán, mô hình được lựa chọn dựa trên chỉ số phù hợp với dữ liệu mất cân bằng, và kết quả đầu ra có thể tích hợp vào hệ thống lưu trữ – cảnh báo – dashboard phục vụ giám sát.

Để đạt mục tiêu tổng quát đó, đề tài hướng tới các mục tiêu cụ thể sau đây. Thứ nhất, khảo sát và mô tả bộ dữ liệu giao dịch tài chính từ Kaggle nhằm làm rõ cấu trúc dữ liệu, các biến quan trọng và những vấn đề dữ liệu cần xử lý. Thứ hai, tích hợp dữ liệu giao dịch với dữ liệu nhãn, làm sạch, biến đổi, tạo đặc trưng và chuẩn bị tập dữ liệu phục vụ mô hình

hóa. Thứ ba, huấn luyện và so sánh các mô hình học máy có giám sát phù hợp với bài toán fraud detection. Thứ tư, đánh giá mô hình theo Precision, Recall, F1-score và ROC-AUC thay vì chỉ dựa vào Accuracy. Thứ năm, thiết kế lớp lưu trữ và lớp trình bày để kết quả mô hình có thể được giám sát gần thời gian thực trên dashboard.

Các câu hỏi nghiên cứu trung tâm của đề tài bao gồm: (1) bộ dữ liệu cần được xử lý như thế nào để phù hợp cho fraud detection; (2) những đặc trưng nào có giá trị giúp mô hình nhận diện fraud tốt hơn; (3) mô hình nào phù hợp hơn trong bối cảnh dữ liệu cực mất cân bằng; (4) ngưỡng cảnh báo nào hợp lý hơn giữa yêu cầu bắt fraud và chi phí rà soát; và (5) mô hình sau khi đánh giá kỹ thuật có thể tạo ra giá trị kinh tế như thế nào cho tổ chức.

1.4. Đối tượng, phạm vi và dữ liệu nghiên cứu

Đối tượng nghiên cứu của đề tài là các giao dịch tài chính trong môi trường thanh toán trực tuyến và bài toán nhận diện giao dịch gian lận dựa trên thông tin giao dịch, khách hàng, thẻ và merchant. Phạm vi nghiên cứu tập trung vào bộ dữ liệu Kaggle computingvictor/transactions-fraud-datasets, trong đó hai nguồn dữ liệu chính được sử dụng là transactions_data.csv và train_fraud_labels.json (Caixabank Tech, 2024).

Về quy mô, file giao dịch chứa 13.305.915 bản ghi, còn file nhãn cung cấp 8.914.963 nhãn hợp lệ. Sau bước inner join theo khóa id, tập master dùng cho phân tích và mô hình hóa có 8.914.963 dòng; 4.390.952 giao dịch không có nhãn bị loại khỏi quy trình học có giám sát. Tỷ lệ fraud trong tập sau tích hợp là 0,1495%, cho thấy mức mất cân bằng rất nghiêm trọng. Đây là cơ sở trực tiếp để xác định phương pháp đánh giá và chiến lược xử lý dữ liệu ở các chương tiếp theo.

Đề tài không hướng tới triển khai một hệ thống sản xuất ở quy mô doanh nghiệp hoàn chỉnh với hạ tầng phân tán đầy đủ, mà tập trung vào một quy trình học thuật có thể thực thi, logic rõ ràng và có giá trị minh họa thực tiễn. Tuy vậy, toàn bộ pipeline vẫn được thiết kế theo hướng có thể mở rộng: dữ liệu sau Chương 2 khớp với thiết kế lưu trữ ở Chương 3 và đầu vào mô hình ở Chương 4, bảo đảm tính nhất quán theo yêu cầu của hướng dẫn báo cáo.

1.5. Phương pháp nghiên cứu

Đề tài sử dụng kết hợp bốn nhóm phương pháp chính. Thứ nhất là nghiên cứu tài liệu nhằm xây dựng nền tảng lý thuyết về fraud detection, dữ liệu mất cân bằng, kỹ thuật đặc

trung, đánh giá mô hình và thiết kế hệ thống giám sát. Thứ hai là phân tích dữ liệu để khám phá cấu trúc bộ dữ liệu, nhận diện missing values, phân bố amount, tương quan giữa các biến và các quy luật theo thời gian. Thứ ba là học máy có giám sát, cụ thể là huấn luyện và so sánh Logistic Regression, Random Forest và XGBoost cùng với GridSearchCV. Thứ tư là phương pháp thiết kế hệ thống dữ liệu lớn, trong đó MongoDB Atlas được dùng làm kho dữ liệu trung tâm, còn Streamlit và ngrok phục vụ lớp hiển thị và công bố demo.

Điểm nhấn phương pháp luận của đề tài là sự liên kết chặt chẽ giữa ba lớp: lớp dữ liệu, lớp mô hình và lớp vận hành. Dữ liệu không được xử lý rời rạc mà phải tạo thành đầu vào có thể dùng được cho mô hình; mô hình không chỉ được đánh giá kỹ thuật mà còn phải chuyển thành đầu ra có thể tích hợp vào dashboard; dashboard không chỉ trình bày số liệu mà còn phải hỗ trợ trả lời câu hỏi về giá trị kinh tế của hệ thống.

1.6. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài góp phần minh họa một quy trình tương đối đầy đủ cho bài toán fraud detection trên dữ liệu giao dịch lớn: từ dữ liệu thô, nhận, xử lý mất cân bằng, feature engineering, huấn luyện mô hình, threshold tuning cho đến thiết kế lớp lưu trữ và giám sát. Điểm có ý nghĩa không nằm ở việc thử thật nhiều thuật toán, mà ở việc giải quyết bài toán như một hệ thống liên ngành giữa dữ liệu, học máy và vận hành.

Về mặt thực tiễn, đề tài cho thấy cách biến dữ liệu giao dịch thành tín hiệu rủi ro có thể phục vụ theo dõi và ưu tiên kiểm tra thủ công. Nếu mô hình được tinh chỉnh phù hợp và được gắn với các giả định chi phí – lợi ích hợp lý, tổ chức có thể ước lượng giá trị kinh tế của hệ thống thay vì dừng lại ở các chỉ số kỹ thuật thuần túy. Đây cũng là tiêu chí cốt lõi mà hướng dẫn báo cáo yêu cầu phải trả lời: mô hình mang về bao nhiêu giá trị cho tổ chức.

Kết thúc Chương 1, có thể xác định rõ rằng đề tài không chỉ nhằm xây dựng một bộ phân loại, mà nhằm xây dựng một pipeline phát hiện gian lận có khả năng nối từ dữ liệu lớn đến giải pháp kinh doanh. Từ nền tảng lý thuyết này, Chương 2 sẽ đi vào phương pháp luận và tiền xử lý dữ liệu để tạo lớp dữ liệu đầu vào nhất quán cho mô hình và hệ thống.

CHƯƠNG 2. PHƯƠNG PHÁP LUẬN VÀ TIỀN XỬ LÝ DỮ LIỆU

2.1. Khám phá dữ liệu (EDA)

2.1.1. Biện luận về chiến lược tối ưu RAM và xử lý theo chunk

Với đề tài phát hiện gian lận trên dữ liệu giao dịch lớn, việc tối ưu RAM ngay từ đầu là yêu cầu kỹ thuật cần thiết để bảo đảm toàn bộ pipeline có thể chạy ổn định trên Google Colab. Nguyên nhân là vì dữ liệu đầu vào có kích thước lớn, trong khi quá trình xử lý không chỉ dừng ở bước đọc dữ liệu mà còn bao gồm EDA, tích hợp dữ liệu, làm sạch, tạo đặc trưng, huấn luyện mô hình và kết nối kết quả sang MongoDB – Streamlit. Nếu nạp toàn bộ dữ liệu vào RAM cùng một lúc, bộ nhớ sẽ nhanh chóng bị quá tải do phát sinh thêm các bảng trung gian và bản sao dữ liệu trong từng bước xử lý. Vì vậy, tối ưu RAM ngay từ dòng lệnh đầu tiên là điều kiện cần để tránh lỗi tràn bộ nhớ, reset runtime và gián đoạn quy trình thực hiện.

Từ yêu cầu đó, đề tài lựa chọn xử lý theo chunk như một giải pháp phù hợp. Thay vì đọc toàn bộ tệp dữ liệu trong một lần, dữ liệu được chia thành nhiều phần nhỏ để đọc và xử lý tuần tự. Cách làm này giúp giảm áp lực bộ nhớ tại từng thời điểm nhưng vẫn bảo đảm có thể quét toàn bộ tập dữ liệu. Đây là lựa chọn đặc biệt hiệu quả trong các bước nặng như thống kê dữ liệu thiếu, ghép dữ liệu giao dịch với nhãn fraud, phát hiện bản ghi trùng lặp và gán cờ outlier.

Như vậy, tối ưu RAM ngay từ đầu và xử lý theo chunk không chỉ là biện pháp tránh lỗi kỹ thuật, mà còn là chiến lược nền tảng giúp đề tài duy trì tính ổn định, khả năng tái lập và hoàn thành đầy đủ các bước xử lý trên dữ liệu lớn.

2.1.2. Mô tả nguồn dữ liệu

Hai nguồn dữ liệu chính lấy từ Kaggle được sử dụng trong đề tài gồm `transactions_data.csv` và `train_fraud_labels.json` (<https://www.kaggle.com/datasets/computingvictor/transactions-fraud-datasets/data>). File `transactions_data.csv` chứa thông tin chi tiết của giao dịch: mã giao dịch, thời gian phát sinh, mã khách hàng, mã thẻ, số tiền, hình thức sử dụng chip, merchant, vị trí merchant, mã ngành hàng (MCC) và thông tin lỗi. File `train_fraud_labels.json` lưu nhãn gian lận dưới cấu

trúc target -> {transaction_id: label}, trong đó label được chuẩn hóa thành fraud_label với hai giá trị 0 và 1.

Ngay từ bước mô tả nguồn dữ liệu, có thể thấy rõ rằng đây là một bộ dữ liệu nhiều thành phần, không phải một bảng đơn giản có thể dùng trực tiếp cho mô hình hóa. Cấu trúc này đặt ra yêu cầu tích hợp dữ liệu trước khi đi vào làm sạch và trích xuất đặc trưng. Đồng thời, quy mô file CSV khoảng 1,2 GB và file JSON khoảng 151,7 MB buộc pipeline phải được thiết kế theo hướng tiết kiệm RAM, chủ yếu bằng chunk processing trên Google Colab.

Bảng 2. 1: Các trường thông tin cốt lõi của bộ dữ liệu

Nhóm thông tin	Biến tiêu biểu	Ý nghĩa
Khóa giao dịch	id	Định danh duy nhất của từng giao dịch và dùng để ghép nhãn
Thời gian	date	Thời điểm phát sinh giao dịch; cơ sở để tạo đặc trưng temporal
Khách hàng/thẻ	client_id, card_id	Phục vụ phân tích hành vi theo khách hàng và theo thẻ
Giá trị giao dịch	amount	Biến số trọng tâm trong fraud detection
Ngữ cảnh giao dịch	use_chip, errors	Phản ánh cách giao dịch được thực hiện và các dấu hiệu bất thường
Merchant và vị trí	merchant_id, merchant_city, merchant_state, zip, mcc	Mô tả đơn vị chấp nhận thanh toán và bối cảnh địa lý/ngành hàng
Nhãn mục tiêu	fraud_label	Biến đích của bài toán phân lớp có giám sát

2.1.3. Kiểm tra cấu trúc dữ liệu và chuẩn hóa kiểu dữ liệu cho EDA

Khi đọc một mẫu 5.000 bản ghi đầu tiên từ transactions_data.csv, dữ liệu ban đầu cho thấy nhiều trường chưa ở đúng kiểu dữ liệu kỳ vọng cho phân tích. Cột date đang ở dạng chuỗi, amount chứa ký hiệu tiền tệ nên cũng được đọc như object, còn zip có missing values nên bị ép thành float. Nếu giữ nguyên cấu trúc này, các thống kê mô tả và biểu đồ sẽ thiếu chính xác, đồng thời pipeline phía sau sẽ phải gánh thêm chi phí chuyển đổi.

Vì vậy, trước khi thực hiện EDA sâu hơn, dữ liệu được chuẩn hóa như sau: date được chuyển sang datetime; amount loại bỏ ký hiệu \$ và dấu phẩy rồi ép về float32; zip được xem như mã, không phải biến liên tục, nên được chuẩn hóa về chuỗi; mcc được giữ dưới dạng số nguyên nhẹ; các biến như use_chip, merchant_state và errors được đổi sang

category để giảm bộ nhớ. Sau chuẩn hóa, bộ nhớ của mẫu 5.000 bản ghi giảm rõ rệt, từ khoảng 468,9 KB xuống 222,7 KB. Điều này là bằng chứng trực tiếp cho thấy việc tối ưu kiểu dữ liệu ngay từ đầu là cần thiết trong bài toán dữ liệu lớn.

Bảng 2. 2: Cấu trúc dữ liệu sau chuẩn hóa trên mẫu EDA

Chỉ tiêu	Giá trị
Số dòng preview	5.000
Số cột preview	12
Số cột số	6
Số cột phân loại/chuỗi	5
Số cột thời gian	1
Dung lượng transactions_data.csv	1.200,23 MB
Dung lượng train_fraud_labels.json	151,71 MB

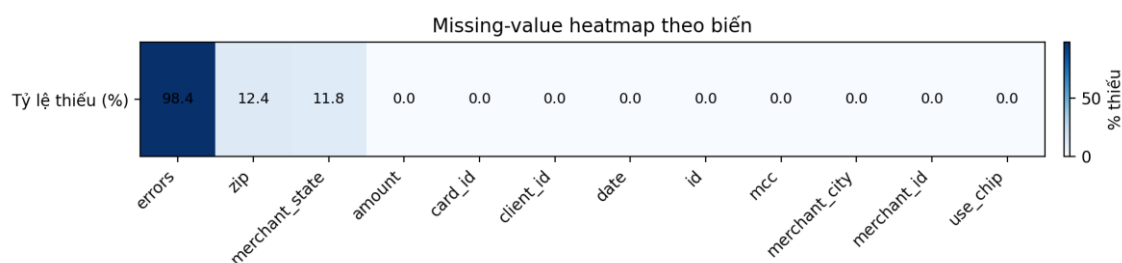
Từ kết quả trên, hướng xử lý theo chunk kết hợp tối ưu RAM được xác lập như một nguyên tắc xuyên suốt cho toàn bộ pipeline. Điều này bảo đảm rằng dữ liệu được xử lý nhất quán từ EDA, tích hợp, làm sạch đến mô hình hóa, thay vì mỗi bước sử dụng một chiến lược độc lập.

2.1.4. Khám phá dữ liệu thiếu

Kết quả thống kê trên toàn bộ tập giao dịch cho thấy dữ liệu thiếu chủ yếu tập trung ở ba cột là errors, zip và merchant_state. Trong đó, errors có tỷ lệ thiếu rất cao, lên tới 98,4113%, trong khi zip và merchant_state lần lượt thiếu 12,4208% và 11,7519%. Các cột còn lại như id, date, amount, client_id, card_id, merchant_id, merchant_city, mcc và use_chip hầu như đầy đủ, cho thấy chất lượng dữ liệu ở các trường lỗi tương đối tốt.

Bảng 2. 3: Tóm tắt dữ liệu thiếu trên transactions_data.csv

Cột	Số lượng thiếu	Tỷ lệ thiếu (%)	Nhận xét
errors	13.094.522	98,4113	Thiếu rất nhiều; có thể mang nghĩa 'không phát sinh lỗi'
zip	1.652.706	12,4208	Thiếu vừa; vẫn có giá trị về mặt địa lý
merchant_state	1.563.700	11,7519	Thiếu vừa; nên giữ lại và gắn cờ missing
Các cột còn lại	0 hoặc gần 0	0,0000	Đủ tốt để giữ nguyên cho pipeline



Hình 2. 1: Ma trận dữ liệu thiếu trên mẫu giao dịch (minh họa cho các cột thiếu chính)

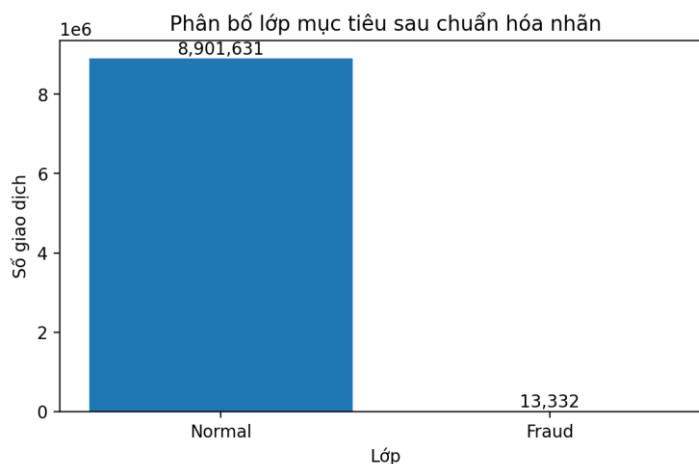
Về ý nghĩa xử lý, ba cột thiếu này không nên được áp dụng cùng một quy tắc. Với errors, giá trị thiếu nhiều khả năng không đơn thuần là lỗi dữ liệu, mà có thể phản ánh trạng thái “không phát sinh lỗi” trong giao dịch. Trong khi đó, zip và merchant_state dù thiếu ở mức vừa nhưng vẫn mang thông tin vị trí có giá trị đối với phân tích gian lận. Vì vậy, hướng xử lý phù hợp là giữ lại các cột này, đồng thời xem xét điền giá trị đại diện và bổ sung cờ is_missing ở các bước làm sạch tiếp theo.

2.1.5. Khám phá phân bố lớp mục tiêu

Phân bố lớp mục tiêu trên toàn bộ file nhận sau khi bung bảng và chuẩn hóa cho thấy 8.901.631 giao dịch thuộc lớp 0 và chỉ 13.332 giao dịch thuộc lớp 1. Tỷ lệ fraud là 0,1495%. Đây là một mức mất cân bằng cực mạnh, xác nhận rằng Accuracy không thể được chọn làm chỉ số trọng tâm cho đề tài.

Bảng 2. 4: Phân bố lớp fraud_label

fraud_label	Số lượng	Tỷ lệ (%)
0 (Normal)	8.901.631	99,8505
1 (Fraud)	13.332	0,1495



Hình 2. 2: Phân bố lớp mục tiêu của bài toán fraud detection

Trong bối cảnh này, nếu mô hình chỉ dự đoán phần lớn giao dịch là bình thường thì vẫn có thể đạt độ chính xác rất cao nhưng gần như không có giá trị thực tế trong phát hiện gian lận. Vì vậy, quá trình đánh giá cần ưu tiên các chỉ số phản ánh tốt hơn khả năng nhận diện lớp hiếm như Recall, Precision, F1-score và ROC-AUC. Kết quả này cũng là cơ sở để định hướng các bước tiếp theo, bao gồm lựa chọn mô hình phù hợp và áp dụng các kỹ thuật xử lý mất cân bằng như class weight, resampling hoặc SMOTE.

2.1.6. Khám phá phân bố amount và ngoại lệ

Trên toàn bộ 13.305.915 quan sát hợp lệ của cột amount, giá trị nhỏ nhất là -500, lớn nhất là 6.820,20; mean bằng 42,976; median bằng 28,99 và độ lệch chuẩn bằng 79,277. Các thống kê này cho thấy amount có phân phối lệch phải mạnh, phần lớn giao dịch tập trung ở vùng thấp nhưng tồn tại một đuôi dài về phía các giao dịch giá trị lớn.

Một đặc điểm đáng chú ý là sự xuất hiện của amount âm. Trên tập master sau tích hợp, có 442.779 giao dịch có giá trị âm. Về mặt nghiệp vụ, đây chưa thể xem là lỗi dữ liệu cần loại bỏ ngay, vì chúng có thể phản ánh hoàn tiền, điều chỉnh hoặc nghiệp vụ tài chính ngược chiều. Do đó, thay vì xóa bỏ, đề tài giữ lại các giao dịch này và bổ sung cờ amount_is_negative để mô hình có thể học được tín hiệu tương ứng.

Khi phát hiện ngoại lệ bằng cả IQR và Z-score trên cột amount, có 706.130 giao dịch bị gán cờ theo IQR và 149.901 giao dịch bị gán cờ theo Z-score. Kết quả này phù hợp với đặc điểm phân phối lệch phải của dữ liệu: IQR nhạy hơn với outlier trong phân phối không chuẩn, còn Z-score chặt hơn vì phụ thuộc vào mean và std. Trong fraud detection, outlier về amount có thể chính là tín hiệu gian lận, nên chiến lược của đề tài là gán cờ thay vì loại bỏ hàng loạt.

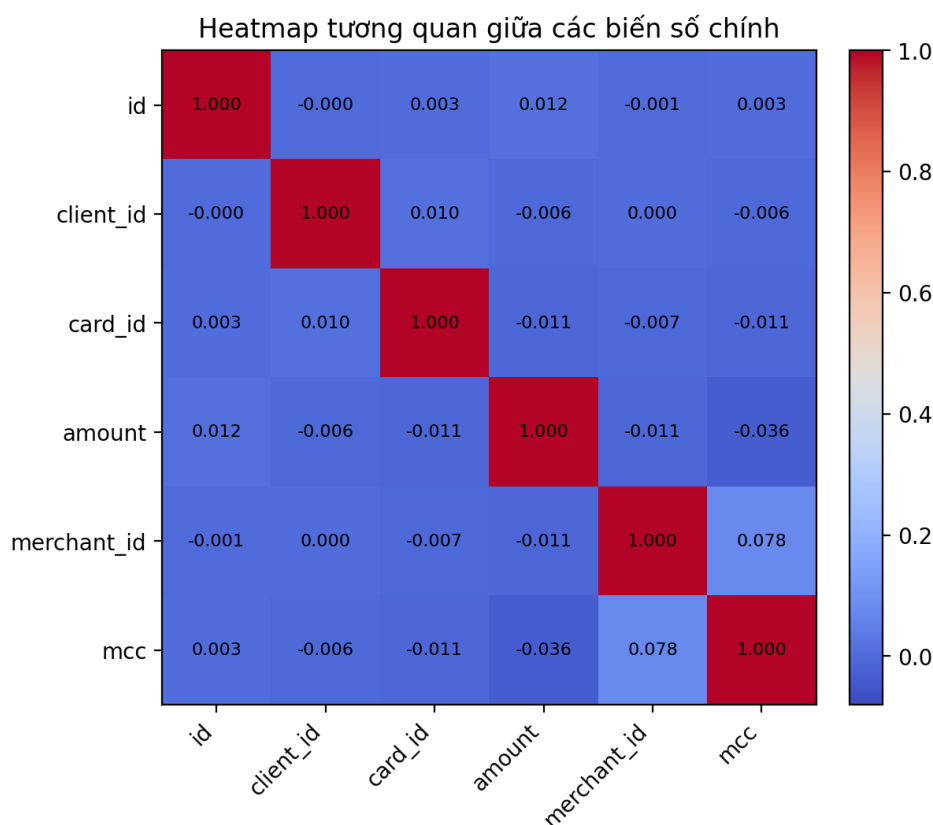
Bảng 2. 5: Kết quả phát hiện outliers trên biến amount

Phương pháp	Số outliers phát hiện
IQR	706.130
Z-score ($ z > 3$)	149.901

2.1.7. Tương quan và phân tích phụ thuộc giữa các biến số

Heatmap tương quan cho thấy các biến số chính trong bộ dữ liệu có mức tương quan tuyến tính rất thấp. Phần lớn các hệ số đều gần bằng 0, cho thấy giữa các biến như client_id, card_id, amount, merchant_id và mcc không tồn tại mối quan hệ tuyến tính mạnh. Riêng

biến id có tương quan gần như bằng 0 với các biến còn lại là điều hợp lý, vì đây chỉ là mã định danh giao dịch, không phản ánh đặc điểm hành vi hay giá trị nghiệp vụ.



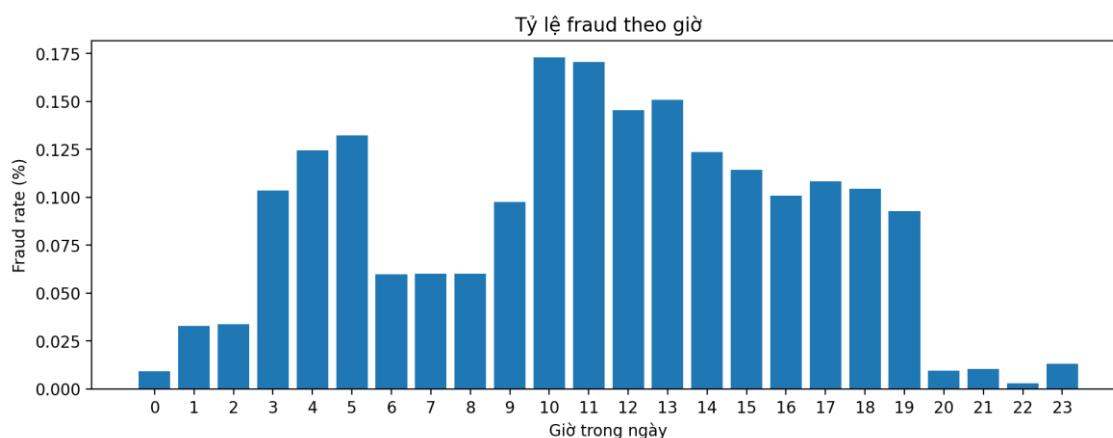
Hình 2. 3: Heatmap tương quan giữa các biến số chính trên mẫu EDA

Trong ma trận tương quan, cặp biến có hệ số lớn nhất là merchant_id và mcc với giá trị khoảng 0,078, nhưng đây vẫn chỉ là mức tương quan rất yếu. Kết quả này cho thấy dữ liệu hiện tại không có dấu hiệu đa cộng tuyến mạnh giữa các biến số chính, tức chưa xuất hiện rủi ro lớn về việc các biến mang thông tin trùng lặp theo nghĩa tuyến tính. Đây là một tín hiệu tích cực trong giai đoạn khám phá dữ liệu.

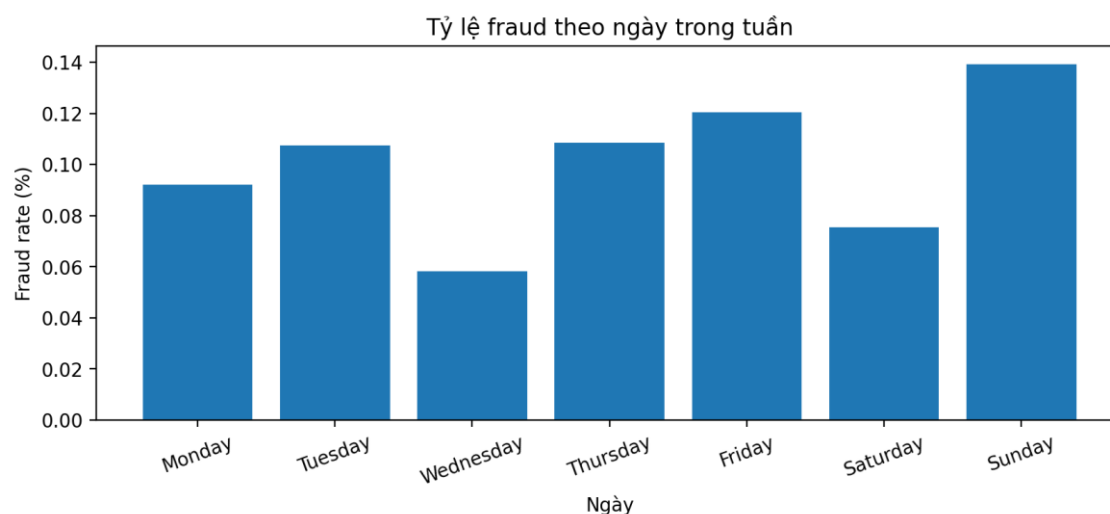
Tuy nhiên, tương quan thấp không đồng nghĩa với việc các biến không có giá trị dự báo. Trong bài toán phát hiện gian lận, nhiều tín hiệu quan trọng thường xuất hiện dưới dạng quan hệ phi tuyến, tương tác giữa nhiều biến hoặc các đặc trưng được tạo thêm trong bước feature engineering. Vì vậy, heatmap này chủ yếu cho thấy các biến số gốc khá độc lập về mặt tuyến tính, còn giá trị thực sự của chúng vẫn cần được đánh giá thêm thông qua các bước xây dựng đặc trưng và huấn luyện mô hình ở phần sau.

2.1.8. Phân tích theo thời gian

Các biểu đồ theo giờ và theo ngày trong tuần cho thấy yếu tố thời gian có ảnh hưởng rõ rệt đến hành vi giao dịch và mức độ rủi ro gian lận. Xét theo giờ, số lượng giao dịch không phân bố đều trong ngày: giao dịch khá thấp vào rạng sáng, tăng mạnh từ sáng và duy trì ở mức cao trong khoảng ban ngày, sau đó giảm dần vào buổi tối. Tuy nhiên, tỷ lệ fraud không biến động hoàn toàn cùng chiều với khối lượng giao dịch. Một số khung giờ như khoảng 10 giờ đến 13 giờ có tỷ lệ fraud cao hơn đáng kể, trong khi nhiều giờ đêm muộn lại có mức fraud thấp. Điều này cho thấy rủi ro gian lận không chỉ phụ thuộc vào số lượng giao dịch, mà còn gắn với đặc điểm hành vi theo từng thời điểm trong ngày.



Hình 2. 4: Phân bố giao dịch và tỷ lệ fraud theo giờ



Hình 2. 5: Phân bố giao dịch và tỷ lệ fraud theo ngày trong tuần

Xét theo ngày trong tuần, số lượng giao dịch giữa các ngày nhìn chung khá đồng đều, cho thấy hoạt động giao dịch diễn ra tương đối ổn định trong suốt tuần. Dù vậy, tỷ lệ fraud

lại có sự khác biệt rõ ràng giữa các ngày. Cụ thể, Sunday là ngày có tỷ lệ fraud cao nhất, tiếp theo là Friday; trong khi Wednesday có mức thấp nhất. Kết quả này cho thấy dù quy mô giao dịch giữa các ngày không chênh lệch lớn, mức độ rủi ro vẫn thay đổi theo chu kỳ tuần, phản ánh khả năng hành vi gian lận có liên hệ với đặc điểm hoạt động của người dùng hoặc merchant vào từng ngày cụ thể.

Từ hai nhóm biểu đồ trên có thể kết luận rằng các biến thời gian là nhóm đặc trưng quan trọng đối với bài toán phát hiện gian lận. Các biến như hour, day_of_week, is_weekend, is_night hoặc các nhóm thời gian trong ngày cần được giữ lại trong bước feature engineering. Việc khai thác temporal features sẽ giúp mô hình nhận diện tốt hơn các mẫu giao dịch bất thường, thay vì chỉ dựa trên các biến gốc như số tiền hay thông tin merchant.

2.2. Quy trình tiền xử lý dữ liệu

2.2.1. Tích hợp đa nguồn dữ liệu (Merge/Join)

Hai file nguồn của đề tài không thể đưa trực tiếp vào mô hình vì một file chứa thông tin giao dịch còn file kia chứa nhãn. Vì vậy, bước đầu tiên của tiền xử lý là tích hợp dữ liệu bằng khóa id. Trước khi merge, khóa ghép ở cả hai nguồn được chuẩn hóa về cùng định dạng string để tránh lỗi do khác kiểu dữ liệu, khoảng trắng hoặc dạng biểu diễn số.

Kết quả kiểm tra cho thấy transactions_data.csv có 13.305.915 id duy nhất, không thiếu và không trùng; train_fraud_labels.json sau khi bung bảng có 8.914.963 id hợp lệ, cũng không thiếu và không trùng. Điều này cho phép sử dụng inner join với validate='many_to_one' một cách an toàn. Việc chọn inner join thay vì left join hay outer join là hoàn toàn phù hợp vì bài toán của đề tài là học có giám sát: chỉ những giao dịch có nhãn mới có thể dùng để huấn luyện và đánh giá mô hình.

Bảng 2. 6: Kết quả sau bước tích hợp dữ liệu

Chỉ tiêu	Giá trị
Số bản ghi transactions_data.csv	13.305.915
Số nhãn hợp lệ trong train_fraud_labels.json	8.914.963
Số bản ghi sau inner join	8.914.963
Số bản ghi bị loại do không có nhãn	4.390.952

Sau inner join, bảng master_transactions có 8.914.963 dòng và 13 cột, bao gồm toàn bộ biến đầu vào từ file giao dịch và biến đích fraud_label từ file nhãn. Đây là đầu vào chính cho các bước làm sạch, feature engineering và mô hình hóa. Tính nhất quán giữa dữ liệu sau tích hợp với dữ liệu của Chương 3 và Chương 4 được bảo đảm ngay từ bước này.

2.2.2. Làm sạch dữ liệu: missing values, dữ liệu nhiễu và trùng lặp

Trên bảng master sau tích hợp, missing values vẫn tập trung ở errors, zip và merchant_state với tỷ lệ gần tương đương giai đoạn EDA. Phương án làm sạch được lựa chọn là điền giá trị kết hợp với gán cờ missing. Với biến số amount, giá trị điền là median 28,99. Với biến phân loại, giá trị điền là mode tương ứng của từng cột. Đồng thời, các cột merchant_state_is_missing, zip_is_missing và errors_is_missing được thêm vào để giữ thông tin về tình trạng thiếu.

Lý do không xóa trực tiếp các bản ghi có missing values là vì điều đó sẽ làm giảm kích thước dữ liệu ở lớp đã vốn hiếm fraud, đồng thời làm mất một dạng tín hiệu có thể hữu ích. Đối với errors, missing còn mang ý nghĩa nghiệp vụ riêng; do đó, chiến lược điền + gán cờ là hợp lý hơn so với xóa bỏ. Đây cũng là cách xử lý nhất quán với mục tiêu giữ lại càng nhiều tín hiệu fraud càng tốt.

Bảng 2. 7: Các quyết định chính trong bước làm sạch

Vấn đề	Phương án	Biện luận
Missing ở amount	Điền median	Giảm mất dữ liệu, ít nhạy với outlier hơn mean
Missing ở biến phân loại	Điền mode	Giữ ổn định phân bố chung của dữ liệu
Missing ở merchant_state/zip/errors	Điền + gán cờ is_missing	Giữ tín hiệu thiếu dữ liệu phục vụ mô hình
Amount âm	Giữ lại + gán cờ	Có thể phản ánh refund/adjustment, không phải lỗi chắc chắn
Outlier amount	Gán cờ IQR/Z-score	Có thể là tín hiệu gian lận, không nên loại bỏ cơ học
Trùng hoàn toàn	Kiểm tra và loại nếu có	Kết quả thực tế: không phát hiện trùng hoàn toàn

Về dữ liệu nhiễu, sau khi chuẩn hóa, không có timestamp lỗi và không có id sai định dạng. Như đã phân tích ở trên, các giao dịch amount âm được giữ lại và gán cờ amount_is_negative. Về trùng lặp, quá trình kiểm tra cho thấy không có bản ghi trùng hoàn

toàn trên tập master; vì vậy, không có dòng nào bị loại ở bước loại trùng hoàn toàn. Kết quả này giúp giữ nguyên quy mô dữ liệu 8.914.963 dòng cho các bước sau.

2.2.3. Biến đổi dữ liệu: chuẩn hóa và mã hóa

Sau bước làm sạch, dữ liệu cần được đưa về định dạng phù hợp cho mô hình học máy. Do tập dữ liệu chứa nhiều biến phân loại có số lượng mức lớn như merchant_id, mcc, client_id, card_id, merchant_city, merchant_state, zip, use_chip và errors, one-hot encoding sẽ làm không gian đặc trưng nở rất mạnh và gây áp lực bộ nhớ không cần thiết. Vì vậy, đề tài sử dụng frequency encoding cho nhóm biến phân loại có nhiều mức. Bản đồ tần suất chỉ được học trên X_train_raw, sau đó được áp dụng cho validation và test, tránh rò rỉ thông tin.

Đối với các biến số liên tục quan trọng như amount, amount_log, client_avg_amt, amt_diff_from_client_avg, client_tx_count và amt_night_interaction, đề tài sử dụng Standard Scaling. Việc fit scaler chỉ trên tập train là bắt buộc để giữ nguyên nguyên tắc đánh giá khách quan. Kết quả của frequency encoding kết hợp standard scaling tạo ra một không gian đặc trưng hoàn toàn dạng số, sẵn sàng cho SMOTE và các mô hình học máy.

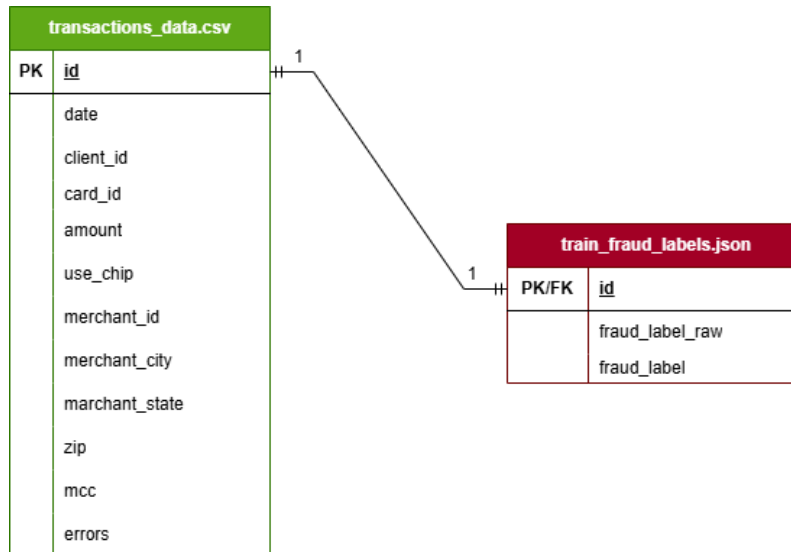
Trong quá trình chuẩn bị dữ liệu mô hình, tập dữ liệu được chia theo chiến lược stratified 60% train, 20% validation, 20% test. Sau đó SMOTE được áp dụng chỉ trên tập huấn luyện với sampling_strategy = 0,1 để cải thiện khả năng học trên lớp fraud mà không làm thay đổi phân bố gốc của validation và test. Đây là một điểm rất quan trọng để bảo đảm các chỉ số ở Chương 4 phản ánh đúng khả năng tổng quát hóa của mô hình trên dữ liệu chưa thấy trước.

Bảng 2. 8: Các bước biến đổi phục vụ mô hình

Thành phần	Phương pháp	Nguyên tắc áp dụng
Biến phân loại nhiều mức	Frequency Encoding	Học mapping trên train, áp dụng cho valid/test
Biến số liên tục	Standard Scaling	Fit scaler trên train, transform trên valid/test
Dữ liệu mất cân bằng ở train	SMOTE (0,1)	Chỉ dùng cho train sau khi chia tập
Chia dữ liệu	Stratified split 60/20/20	Giữ ổn định tỷ lệ fraud ở các tập

2.2.4. Sơ đồ quan hệ ERD

Để làm rõ cơ sở của bước tích hợp dữ liệu, đề tài xây dựng sơ đồ quan hệ giữa hai nguồn dữ liệu chính là `transactions_data.csv` và `train_fraud_labels.json`. Sơ đồ này cho thấy trường `id` giữ vai trò khóa liên kết giữa dữ liệu giao dịch và dữ liệu nhãn, từ đó hình thành tập dữ liệu đầu vào hoàn chỉnh cho bài toán phân lớp có giám sát.



Hình 2. 6: Sơ đồ dữ liệu ERD

Trong sơ đồ, `transactions_data.csv` là bảng dữ liệu giao dịch gốc, chứa các thuộc tính mô tả từng giao dịch như thời gian giao dịch (`date`), mã khách hàng (`client_id`), mã thẻ (`card_id`), số tiền (`amount`), hình thức sử dụng chip (`use_chip`), thông tin merchant (`merchant_id`, `merchant_city`, `merchant_state`, `zip`, `mcc`) và thông tin lỗi (`errors`). Trường `id` trong bảng này đóng vai trò khóa chính, dùng để định danh duy nhất mỗi giao dịch. Trong khi đó, `train_fraud_labels.json` là nguồn dữ liệu nhãn, trong đó mỗi `id` giao dịch được gắn với một giá trị nhãn ban đầu (`fraud_label_raw`) và một nhãn đã chuẩn hóa (`fraud_label`) phục vụ cho mô hình học máy. Trong cấu trúc quan hệ này, `id` của bảng nhãn vừa là khóa chính vừa là khóa ngoại tham chiếu đến `id` trong bảng giao dịch.

Sơ đồ thể hiện mối quan hệ 1–1 giữa hai nguồn dữ liệu theo trường `id`, nghĩa là mỗi giao dịch được ghép với đúng một nhãn gian lận tương ứng. Quan hệ này phù hợp với bản chất của bài toán học có giám sát, trong đó mỗi quan sát đầu vào phải đi kèm đúng một biến mục tiêu. Từ quan hệ này, đề tài thực hiện phép `inner join` để chỉ giữ lại những giao dịch đồng thời xuất hiện ở cả hai nguồn dữ liệu, tức là vừa có thông tin giao

dịch, vừa có nhãn fraud. Kết quả của bước này là bảng master_transactions, đóng vai trò là tập dữ liệu trung tâm cho các bước làm sạch, tạo đặc trưng và huấn luyện mô hình ở các phần tiếp theo.

2.3. Kỹ thuật đặc trưng

Sau khi dữ liệu được tích hợp và làm sạch, bước tiếp theo là tạo đặc trưng mới nhằm tăng khả năng nhận diện gian lận của mô hình. Theo yêu cầu của hướng dẫn báo cáo, các đặc trưng được tạo ra phải có giá trị kinh tế hoặc hành vi, nghĩa là có thể giải thích tại sao chúng liên quan đến rủi ro fraud chứ không chỉ là phép biến đổi hình thức.

2.3.1. Đặc trưng thời gian

Nhóm temporal features gồm hour, day_of_week, is_weekend và is_night. Đây là nhóm đặc trưng trực tiếp xuất phát từ các kết quả EDA theo thời gian. Hour giúp mô hình nhận diện khung giờ giao dịch; day_of_week mô tả chu kỳ tuần; is_weekend gom nhóm thứ Bảy và Chủ nhật để kiểm tra hiệu ứng cuối tuần; is_night được xây dựng để nêu bật các giao dịch diễn ra trong khoảng thời gian nhạy cảm. Nhóm này đặc biệt quan trọng trong fraud detection vì rủi ro không phân bố đều theo thời gian.

2.3.2. Đặc trưng tương tác

Đề tài xây dựng amount_log để làm mềm ảnh hưởng của các giao dịch giá trị rất lớn, đồng thời giữ lại thứ bậc tương đối của amount. Bên cạnh đó, amt_night_interaction = amount $\times$ is_night giúp nêu bật các giao dịch có số tiền lớn xuất hiện vào giờ đêm. Loại đặc trưng này có ý nghĩa vì cùng một amount, nếu xuất hiện vào thời điểm bất thường thì mức độ nghi ngờ có thể cao hơn.

2.3.3. Đặc trưng tích lũy theo khách hàng

Một hạn chế của dữ liệu giao dịch thô là không phản ánh được bối cảnh lịch sử của từng khách hàng. Để khắc phục điều này, đề tài bổ sung client_avg_amt, amt_diff_from_client_avg và client_tx_count. client_avg_amt cho biết mức giao dịch trung bình lịch sử của khách hàng; amt_diff_from_client_avg phản ánh giao dịch hiện tại lệch bao nhiêu so với nền hành vi của chính khách hàng đó; client_tx_count mô tả độ dày lịch sử giao dịch. Những đặc trưng này giúp mô hình phân biệt giữa một amount vốn bình thường với khách hàng A nhưng lại rất bất thường với khách hàng B.

Bảng 2. 9: Các đặc trưng được tạo thêm ở bước feature engineering

Nhóm đặc trưng	Biến	Ý nghĩa kinh tế/hành vi
Thời gian	hour, day_of_week	Phản ánh chu kỳ giao dịch trong ngày và trong tuần
Thời gian nhị phân	is_weekend, is_night	Nêu bật các bối cảnh thời gian nhạy cảm
Biến đổi số tiền	amount_log	Giảm lệch phải của amount
Tương tác	amt_night_interaction	Nhấn mạnh giao dịch lớn vào thời điểm bất thường
Tích lũy khách hàng	client_avg_amt	Chuẩn hành vi giao dịch lịch sử của khách hàng
Độ lệch hành vi	amt_diff_from_client_avg	Mức bất thường so với lịch sử cá nhân
Độ dày lịch sử	client_tx_count	Mức độ ổn định của hành vi khách hàng

Sau bước feature engineering, file master_features.csv được tạo ra như lớp dữ liệu đầu vào trực tiếp cho Chương 4 và đồng thời là nguồn quan trọng cho thiết kế schema trong Chương 3. Việc cùng một tập đặc trưng được dùng nhất quán cho mô hình và hệ thống là yêu cầu rất quan trọng của hướng dẫn báo cáo, và cũng là nguyên tắc phương pháp luận mà đề tài tuân thủ.

2.4. Kết luận chương 2

Chương 2 đã hoàn thành mục tiêu tạo ra lớp dữ liệu nhất quán cho toàn bộ đề tài. Thông qua EDA, nghiên cứu xác định rõ đặc trưng dữ liệu lớn, mức độ mất cân bằng của fraud, các vấn đề missing values, outliers, biến động theo thời gian và nhu cầu tối ưu RAM. Trên cơ sở đó, pipeline tiền xử lý được xây dựng gồm: tích hợp hai nguồn dữ liệu bằng inner join; làm sạch với chiến lược điền giá trị kết hợp gán cờ; chuẩn hóa và mã hóa dữ liệu theo hướng phù hợp cho mô hình; tạo đặc trưng thời gian, tương tác và tích lũy theo khách hàng.

Kết quả cuối cùng của chương là hai lớp dữ liệu chính: master_transactions_cleaned_flagged.csv và master_features.csv. Lớp thứ nhất lưu dữ liệu sạch cùng các cờ chất lượng dữ liệu; lớp thứ hai lưu dữ liệu đã tạo đặc trưng, sẵn sàng cho mô hình hóa. Đây chính là cầu nối giữa bài toán dữ liệu của Chương 2 với bài toán hạ tầng ở Chương 3 và bài toán khai phá ở Chương 4.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG DỮ LIỆU LỚN

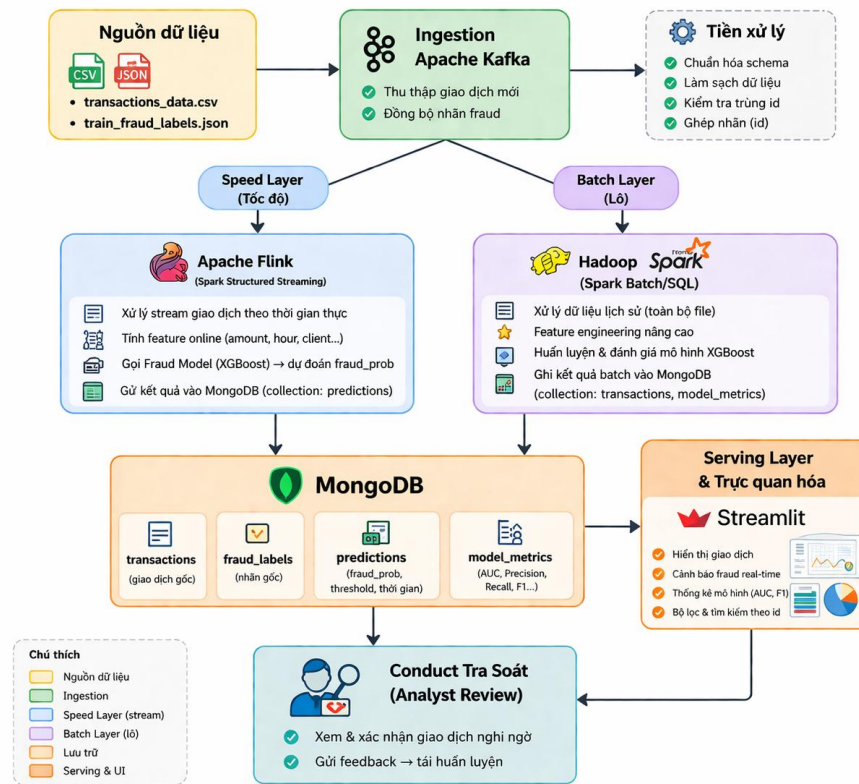
3.1. Kiến trúc hệ thống

Theo yêu cầu của đề bài, Chương 3 phải thể hiện rõ luồng đi từ dữ liệu đã được xử lý sang hạ tầng lưu trữ, cơ chế khai phá và lớp giải pháp kinh doanh. Trong đề tài này, kiến trúc hệ thống được thiết kế theo tư duy Lambda Architecture ở mức logic. Dữ liệu lịch sử sau tiên xử lý đi theo batch path để tạo lớp nền ổn định; dữ liệu mới hoặc dữ liệu mô phỏng gần thời gian thực đi theo speed path để tạo dự đoán và cảnh báo nhanh; hai luồng này hội tụ tại serving layer để phục vụ dashboard.

3.1.1. Biện luận lựa chọn kiến trúc

Một hệ thống phát hiện gian lận không chỉ cần mô hình tốt mà còn cần dòng chảy dữ liệu rõ ràng. Nếu chỉ lưu dữ liệu đầu vào và chạy mô hình một lần ngoại tuyến, tổ chức sẽ không có khả năng theo dõi lịch sử dự đoán, cảnh báo mới, phân tầng rủi ro hoặc giá trị kinh tế theo thời gian. Vì vậy, kiến trúc của đề tài được xây dựng theo bốn khối: data ingestion, storage layer, processing layer và serving layer. Cấu trúc này giúp tách biệt trách nhiệm của từng lớp và làm rõ mối liên hệ giữa Chương 2, Chương 3 và Chương 4.

Kiến trúc Lambda cho hệ thống phát hiện gian lận (Theo bài toán của bạn)



Hình 3. 1: Kiến trúc logic của hệ thống phát hiện gian lận trên dữ liệu giao dịch lớn

Từ góc nhìn học thuật, việc mô phỏng một kiến trúc dạng Lambda là phù hợp vì nó cho thấy hệ thống đã được xem xét theo hướng xử lý cả dữ liệu lịch sử lẫn dữ liệu gần thời gian thực. Từ góc nhìn thực thi, kiến trúc được giản lược vừa đủ để có thể triển khai trên Google Colab, MongoDB Atlas, Streamlit và ngrok mà không vượt quá giới hạn tài nguyên của môi trường nghiên cứu.

3.1.2. Các lớp thành phần của hệ thống

Khối 1 – Data Ingestion nhận dữ liệu lịch sử đã được tiền xử lý từ Chương 2, đồng thời có khả năng nhận các batch giao dịch mới hoặc các giao dịch mô phỏng để phục vụ dashboard gần thời gian thực. Khối 2 – Storage Layer sử dụng MongoDB Atlas làm kho dữ liệu trung tâm, nơi lưu cả dữ liệu giao dịch sạch, dữ liệu đặc trưng, dữ liệu dự đoán, dữ liệu cảnh báo và các chỉ số tổng hợp. Khối 3 – Processing Layer được đặt trên Google Colab, nơi thực hiện nạp dữ liệu, tiền xử lý bổ sung, suy luận mô hình, cập nhật summary và đồng bộ dữ liệu lên MongoDB. Khối 4 – Serving Layer sử dụng Streamlit để hiển thị dashboard và ngrok để công bố demo ra ngoài Internet.

Cách phân lớp này giúp mỗi thành phần có vai trò rõ ràng. MongoDB Atlas không chỉ đơn thuần là nơi lưu dữ liệu, mà là lớp trung gian để kết nối kết quả kỹ thuật của mô hình với giao diện giám sát. Streamlit không trực tiếp xử lý dữ liệu gốc, mà chỉ đọc các collection đã được tổng hợp trong MongoDB. Nhờ đó, dashboard hoạt động như một lớp trình bày cuối cùng, phản ánh trạng thái hệ thống một cách mạch lạc thay vì làm nhiệm vụ tính toán nặng.

3.1.3. Luồng dữ liệu batch và speed path

Luồng dữ liệu của hệ thống bắt đầu từ hai nguồn đầu vào chính là transactions_data.csv và train_fraud_labels.json. Trong đó, transactions_data.csv chứa dữ liệu giao dịch gốc, còn train_fraud_labels.json chứa nhãn gian lận tương ứng theo khóa id. Hai nguồn dữ liệu này trước hết được đưa vào lớp Ingestion thông qua Apache Kafka. Tại đây, hệ thống thực hiện chức năng thu nhận giao dịch mới, đồng bộ nhãn fraud và chuyển dữ liệu sang bước tiền xử lý. Khối Tiền xử lý có nhiệm vụ chuẩn hóa schema, làm sạch dữ liệu, kiểm tra trùng lặp theo id và ghép nhãn với giao dịch, nhằm bảo đảm dữ liệu đầu vào đạt tính nhất quán trước khi đi vào các lớp xử lý tiếp theo.

Sau bước tiền xử lý, dữ liệu được tách thành hai nhánh theo đúng nguyên lý của kiến trúc Lambda. Nhánh thứ nhất là Speed Layer, dùng để xử lý dữ liệu theo thời gian thực. Tại đây, hệ thống tiếp nhận luồng giao dịch mới, tính nhanh một số đặc trưng online như số tiền, giờ giao dịch hoặc thông tin khách hàng, sau đó gọi mô hình fraud đã huấn luyện để dự đoán xác suất gian lận (fraud_prob). Kết quả suy luận thời gian thực được ghi trực tiếp vào MongoDB, chủ yếu trong collection predictions, để phục vụ cảnh báo nhanh và hiển thị gần thời gian thực trên dashboard. Nhánh thứ hai là Batch Layer, dùng để xử lý dữ liệu lịch sử theo lô. Dữ liệu toàn bộ tệp được đưa qua môi trường Hadoop/Spark để thực hiện feature engineering nâng cao, huấn luyện và đánh giá mô hình XGBoost, sau đó ghi kết quả batch vào MongoDB dưới dạng dữ liệu giao dịch đã chuẩn hóa và các chỉ số mô hình như AUC, Precision, Recall, F1-score.

MongoDB đóng vai trò là lớp lưu trữ trung tâm của toàn hệ thống. Tại đây, dữ liệu được tổ chức thành các collection chính như transactions để lưu giao dịch gốc, fraud_labels để lưu nhãn, predictions để lưu kết quả dự đoán và model_metrics để lưu các chỉ số đánh giá mô hình. Từ MongoDB, dữ liệu được cung cấp cho Serving Layer và lớp trực quan hóa

thông qua Streamlit. Dashboard có chức năng hiển thị danh sách giao dịch, cảnh báo fraud theo thời gian thực, thống kê mô hình và hỗ trợ bộ lọc hoặc truy vấn theo id. Nhờ đó, người dùng có thể theo dõi trạng thái hệ thống và nhận diện nhanh các giao dịch nghi ngờ.

Ở bước cuối, hệ thống hỗ trợ Conduct Tra Soát (Analyst Review), tức là khâu rà soát nghiệp vụ bởi chuyên viên phân tích. Từ dashboard hoặc trực tiếp từ MongoDB, chuyên viên có thể xem, xác nhận hoặc loại bỏ các giao dịch bị gán cờ. Kết quả rà soát này không chỉ phục vụ vận hành hiện tại mà còn tạo thành một vòng phản hồi (feedback loop) để tái huấn luyện mô hình trong các chu kỳ sau. Như vậy, toàn bộ luồng dữ liệu được tổ chức theo hướng vừa đáp ứng yêu cầu xử lý thời gian thực, vừa bảo đảm khả năng phân tích sâu trên dữ liệu lịch sử, qua đó phù hợp với bài toán phát hiện gian lận trong môi trường dữ liệu lớn.

3.2. Thiết kế lưu trữ (Schema Design)

Lớp lưu trữ được thiết kế theo mô hình document của MongoDB Atlas. Lý do chính của lựa chọn này là dữ liệu đầu ra của hệ thống có cấu trúc bán linh hoạt: ngoài các trường giao dịch và đặc trưng cố định, hệ thống còn cần lưu metadata của mô hình, threshold, alert_level, status_review, prediction_time và các summary theo thời gian. Kiểu document cho phép bổ sung các trường như vậy mà không cần thay đổi schema cứng như trong mô hình quan hệ truyền thống.

3.2.1. Nguyên tắc thiết kế schema

Thiết kế schema của đề tài tuân theo bốn nguyên tắc. Thứ nhất, schema phải bám sát dữ liệu thực của Chương 2, tức dữ liệu được làm sạch và dữ liệu đã tạo đặc trưng phải được phản ánh trực tiếp trong document. Thứ hai, schema phải phục vụ truy vấn dashboard, nghĩa là các trường thường dùng như id, date, fraud_label, fraud_score, prediction_time và status_review phải dễ truy xuất. Thứ ba, schema phải tách rõ dữ liệu nền nghiệp vụ với dữ liệu suy luận và dữ liệu tổng hợp. Thứ tư, schema phải đủ linh hoạt để mở rộng khi thêm phiên bản mô hình, ngưỡng vận hành hay các trường đánh giá trong tương lai.

Bảng 3. 1: Các collection chính của hệ thống MongoDB

Collection	Vai trò chính	Nguồn hình thành
processed_transactions	Lưu dữ liệu giao dịch sạch và cờ chất lượng dữ liệu	master_transactions_cleaned_flagged.csv
master_features	Lưu dữ liệu sau feature engineering	master_features.csv
model_predictions	Lưu lịch sử dự đoán của mô hình	Kết quả suy luận từ mô hình
realtime_alerts	Lưu tập giao dịch vượt ngưỡng cảnh báo	Trích từ model_predictions
dashboard_summary	Lưu các chỉ số tổng hợp phục vụ dashboard	Tổng hợp từ nhiều collection
model_metadata	Lưu metadata của mô hình đang triển khai	Kết quả huấn luyện và đánh giá

3.2.2. Cấu trúc document mẫu

Collection `processed_transactions` phản ánh trực tiếp dữ liệu từ file `master_transactions_cleaned_flagged.csv`. Document của collection này chứa thông tin giao dịch gốc, nhãn gian lận và các cờ chất lượng dữ liệu như `merchant_state_is_missing`, `zip_is_missing`, `errors_is_missing`, `amount_is_negative`, `outlier_iqr_flag` và `outlier_zscore_flag`. Đây là lớp dữ liệu nền để tra cứu lịch sử và phục vụ tổng hợp dashboard.

Collection `master_features` được xây dựng từ `master_features.csv`, vì vậy ngoài các trường giao dịch cơ bản còn có thêm `hour`, `day_of_week`, `is_weekend`, `is_night`, `amount_log`, `amt_night_interaction`, `client_avg_amt`, `amt_diff_from_client_avg` và `client_tx_count`. Collection này là cầu nối giữa lớp dữ liệu sạch và lớp mô hình hóa.

Collection `model_predictions` lưu `fraud_score`, `predicted_label`, `prediction_time`, `model_name` và `model_version`. Collection `realtime_alerts` lấy tập con các giao dịch có `predicted_label = 1`, đồng thời bổ sung `alert_level` và `status_review` để phục vụ xử lý trên dashboard. Collection `dashboard_summary` lưu các chỉ số tổng hợp như tổng giao dịch, số giao dịch gần cờ, tỷ lệ rủi ro, số cảnh báo mới, phân bố fraud theo giờ/ngày và các merchant rủi ro cao. Cuối cùng, `model_metadata` lưu `threshold`, `precision`, `recall`, `f1_score`, `auc` và các ghi chú quản trị của mô hình đang triển khai.

Bảng 3. 2: Ví dụ các trường cốt lõi trong từng collection

Collection	Các trường tiêu biểu
processed_transactions	_id, id, date, amount, use_chip, merchant_id, fraud_label, *_is_missing, outlier_*
master_features	_id, id, date, hour, day_of_week, is_weekend, is_night, amount_log, client_avg_amt
model_predictions	_id, id, fraud_score, predicted_label, prediction_time, model_name, model_version
realtime_alerts	_id, id, amount, fraud_score, predicted_label, alert_level, prediction_time, status_review
dashboard_summary	_id, metric_name, summary_time, total_transactions, flagged_transactions, fraud_ratio, new_alerts
model_metadata	_id, model_name, model_version, training_date, threshold, precision, recall, f1_score, auc

3.2.3. Chỉ mục và tối ưu truy vấn

Để schema không chỉ đúng về mặt cấu trúc mà còn phù hợp với truy vấn, đề tài thiết kế chỉ mục cho từng collection. Với processed_transactions, các trường được ưu tiên là date, fraud_label, merchant_id và client_id nhằm hỗ trợ tra cứu lịch sử, lọc theo thời gian và thống kê theo merchant/khách hàng. Với master_features, ngoài date và fraud_label còn cần hour và day_of_week để phục vụ phân tích theo thời gian.

Với model_predictions, các chỉ mục quan trọng gồm id, prediction_time, fraud_score và predicted_label để truy vết lịch sử dự đoán và sắp xếp theo mức rủi ro. Với realtime_alerts, các chỉ mục quan trọng gồm prediction_time, fraud_score, alert_level và status_review nhằm phục vụ dashboard và quy trình rà soát. Với dashboard_summary, metric_name và summary_time giúp truy xuất nhanh số liệu tổng hợp mới nhất. Với model_metadata, model_name, model_version và training_date giúp quản trị vòng đời mô hình.

Biện luận cốt lõi ở đây là: NoSQL không được chọn chỉ vì tính mới, mà vì kiểu document phù hợp với bản chất dữ liệu của đề tài. Nếu dùng SQL, nhiều bảng và nhiều join phụ sẽ phát sinh thêm độ phức tạp trong truy vấn dashboard, trong khi dữ liệu đầu ra của hệ thống lại có cấu trúc tự nhiên dạng document. Vì vậy, MongoDB Atlas là lựa chọn hợp lý hơn cho một hệ thống giám sát fraud có lớp dự đoán, cảnh báo và metadata động.

3.3. Tính toán hạ tầng lưu trữ

Một yêu cầu quan trọng của Chương 3 là tính toán dung lượng lưu trữ dựa trên cơ chế replication factor. Mục tiêu không phải để tối ưu hạ tầng sản xuất ở mức chi tiết, mà chứng minh rằng hệ thống đã được xem xét từ góc độ mở rộng và khả năng triển khai thực tế.

3.3.1. Cơ sở tính toán

Từ các file dữ liệu đã tạo ở Chương 2, số dòng của processed_transactions là 8.914.963 với kích thước file 1.106,46 MB; số dòng của master_features cũng là 8.914.963 với kích thước file 1.655,14 MB. Trung bình mỗi dòng của processed_transactions chiếm khoảng 130,14 bytes; mỗi dòng của master_features chiếm khoảng 194,68 bytes. Ngoài ra, đề tài giả định kích thước trung bình cho một document prediction là 180 bytes, một document alert là 220 bytes, một document summary là 50.000 bytes và metadata là 2.000 bytes. Tỷ lệ overhead chỉ mục được giả định là 20%.

Bảng 3. 3: Các tham số cơ sở để ước tính lưu trữ

Hạng mục	Giá trị
Số dòng processed_transactions	8.914.963
Số dòng master_features	8.914.963
Kích thước processed_transactions	1.106,46 MB
Kích thước master_features	1.655,14 MB
Bytes/dòng processed_transactions	130,14
Bytes/dòng master_features	194,68
Index overhead ratio	20%

3.3.2. Kết quả ước tính

Bảng 3. 4: Kích bản dung lượng lưu trữ trên MongoDB Atlas

Kịch bản	Prediction rows	Alert rows	RF	Dung lượng trước replication (GB)	Dung lượng sau replication (GB)
Quy mô hiện tại	100.000	1.000	1	3,2567	3,2567
Mở rộng số batch dự đoán	500.000	5.000	1	3,3381	3,3381
Mở rộng + replication factor = 3	500.000	5.000	3	3,3381	10,0144

Ở kịch bản 1, tổng dung lượng sau khi tính overhead chỉ mục khoảng 3,2567 GB. Ở kịch bản 2, dù số dòng prediction và alert tăng mạnh, tổng dung lượng chỉ tăng nhẹ lên 3,3381 GB vì phần lớn dung lượng vẫn đến từ dữ liệu processed_transactions và

master_features. Ở kịch bản 3, khi áp dụng replication factor = 3, tổng dung lượng tăng lên khoảng 10,0144 GB.

Các con số trên cho thấy hệ thống hiện tại nằm trong ngưỡng quản lý được của MongoDB Atlas. Điều này cũng là cơ sở để biện luận rằng ở quy mô đồ án, MongoDB Atlas hoàn toàn có thể đóng vai trò kho dữ liệu trung tâm mà chưa cần sharding hay một kiến trúc phân tán sâu hơn. Replication factor được trình bày như một góc nhìn mở rộng hạ tầng: ngay cả khi cần nâng tính sẵn sàng và an toàn dữ liệu, quy mô dung lượng vẫn còn khả thi.

3.3.3. Pipeline nạp dữ liệu và thử nghiệm trên Google Colab

Thay vì nạp toàn bộ hơn 8,9 triệu dòng lên MongoDB Atlas ngay từ đầu, đề tài chọn chiến lược nạp thử nghiệm một mẫu ngẫu nhiên 25.000 dòng cho processed_transactions và 25.000 dòng cho master_features. Cách tiếp cận này phù hợp với giai đoạn kiểm tra ban đầu vì giúp xác nhận schema, kiểm tra kết nối, kiểm tra collection và quan sát cách document được tổ chức sau khi insert mà không tốn quá nhiều tài nguyên.

Kết quả thử nghiệm cho thấy mỗi collection đều ghi được 25.000 document. Sau khi insert, các collection có thể được truy vấn bình thường, chứng minh rằng pipeline chuyển từ DataFrame sang MongoDB document hoạt động ổn định. Một khi schema và cách nạp mẫu đã được xác nhận, cùng pipeline này có thể mở rộng ra để đồng bộ dữ liệu lớn hơn hoặc toàn bộ dữ liệu nếu cần.

3.3.4. Biện luận lựa chọn MongoDB Atlas thay vì SQL

Từ góc nhìn truy vấn, dashboard cần đồng thời đọc dữ liệu lịch sử, dữ liệu dự đoán, dữ liệu cảnh báo và dữ liệu tổng hợp. Nếu mô hình hóa tất cả theo SQL, hệ thống sẽ cần nhiều bảng, nhiều quan hệ và nhiều truy vấn join phụ. Trong khi đó, các collection của MongoDB cho phép lưu document bám sát đúng bản chất đối tượng nghiệp vụ: giao dịch sạch, đặc trưng, prediction, alert, summary và metadata.

Từ góc nhìn mở rộng, NoSQL còn linh hoạt hơn khi hệ thống cần bổ sung thêm model_version, threshold, alert_level, status_review hay các trường đánh giá khác trong tương lai. Vì vậy, việc chọn MongoDB Atlas không chỉ phù hợp với quy mô đồ án mà còn phù hợp với đặc điểm dữ liệu và hướng triển khai của một hệ thống giám sát fraud.

3.4. Dashboard giám sát gian lận và công bố qua ngrok

3.4.1. Mục tiêu thiết kế dashboard

Dashboard trong đề tài không được thiết kế như một lớp trình diễn tách rời khỏi hệ thống, mà là thành phần cuối của chuỗi xử lý dữ liệu từ Chương 2 đến Chương 4. Sau khi dữ liệu giao dịch được làm sạch, tạo đặc trưng, lưu vào MongoDB Atlas và sinh ra kết quả dự đoán, dashboard đảm nhận chức năng chuyển các tín hiệu kỹ thuật đó thành thông tin có thể quan sát, tra cứu và sử dụng cho quyết định vận hành.

Về mặt mục tiêu, dashboard được xây dựng để đạt bốn yêu cầu đồng thời. Thứ nhất, cung cấp cho người dùng một ảnh chụp tức thời về trạng thái hệ thống thông qua các chỉ số KPI cốt lõi. Thứ hai, cho phép quan sát luồng giao dịch mới theo chu kỳ ngắn nhằm mô phỏng bối cảnh gần thời gian thực. Thứ ba, hỗ trợ rà soát thủ công các cảnh báo có rủi ro cao thông qua cơ chế cập nhật trạng thái xử lý. Thứ tư, tạo lớp trực quan hóa giúp kết nối kết quả mô hình với các đối tượng nghiệp vụ như merchant, số tiền giao dịch và mức độ rủi ro.

Như vậy, mục tiêu của dashboard không chỉ là "hiển thị đẹp" mà là tạo ra một giao diện vận hành tối giản nhưng đủ chức năng để chứng minh tính khả thi của toàn bộ kiến trúc phát hiện gian lận: dữ liệu được cập nhật, cảnh báo được sinh ra, trạng thái rà soát được lưu lại và các chỉ số tổng hợp được phản ánh nhất quán trên cùng một giao diện.

3.4.2. Vai trò của dashboard trong kiến trúc hệ thống

Nếu xem toàn bộ hệ thống theo logic Lambda Architecture đã trình bày ở các mục trước, dashboard thuộc lớp serving layer. Nó không thực hiện các tác vụ nặng như join dữ liệu lớn, huấn luyện mô hình hay tính toán đặc trưng hàng loạt. Thay vào đó, dashboard truy vấn dữ liệu đã được chuẩn hóa và tổng hợp sẵn từ MongoDB Atlas, sau đó biểu diễn lại bằng bảng, biểu đồ và KPI.

Cách tổ chức này mang lại hai lợi ích quan trọng. Một mặt, nó tách biệt rõ trách nhiệm giữa lớp xử lý và lớp trình bày, nhờ đó giao diện giữ được tốc độ phản hồi tốt ngay cả khi dữ liệu nền có kích thước lớn. Mặt khác, nó cho phép dashboard kế thừa trực tiếp trạng thái mới nhất của hệ thống: số lượng giao dịch, số cảnh báo, tỷ lệ rủi ro, danh sách giao dịch chờ rà soát và các điểm bất thường theo merchant hoặc số tiền.

Trong phạm vi đề án, dashboard vì vậy được xem như cửa sổ vận hành của mô hình. Chất lượng của dashboard được đánh giá không chỉ ở tính trực quan, mà còn ở khả năng phản ánh đúng luồng dữ liệu và cho phép người dùng hành động trên kết quả cảnh báo.



Hình 3. 2: Sơ đồ bố cục logic của dashboard giám sát gian lận

Hình 3.4.1 tổng hợp lại bố cục dashboard theo đúng logic triển khai trong mã nguồn. Cấu trúc này gồm sidebar cấu hình ở bên trái, khối header và KPI ở đầu trang, dải tab chức năng ở giữa và vùng bảng – biểu đồ ở phần thân. Việc phân tầng rõ ràng này giúp người dùng đi từ cái nhìn khái quát tới chi tiết mà không phải rời khỏi cùng một giao diện.

3.4.3. Nguồn dữ liệu cấp cho dashboard và logic truy vấn

Khác với cách tiếp cận đọc trực tiếp từ tệp CSV, dashboard của đề tài truy vấn dữ liệu từ các collection đã được tổ chức trước trong MongoDB Atlas. Quyết định này giúp giao diện hoạt động đúng vai trò của một lớp giám sát, đồng thời bảo đảm rằng dữ liệu dùng cho hiển thị, dữ liệu dùng cho mô hình và dữ liệu dùng cho rà soát thuộc cùng một nguồn chân lý.

Các collection được sử dụng cho dashboard gồm: `simulated_live_transactions` để hiển thị batch giao dịch mới nhất; `model_predictions` để phục vụ tra cứu lịch sử dự đoán; `realtime_alerts` để theo dõi tập giao dịch vượt ngưỡng cảnh báo; `dashboard_summary` để lưu các chỉ số tổng hợp nhằm tăng tốc truy vấn; và `model_metadata` để hiển thị thông tin quản trị mô hình như tên mô hình, phiên bản và ngưỡng sử dụng.

Bảng 3. 5: Mapping giữa thành phần giao diện và collection MongoDB

Thành phần dashboard	Collection truy vấn	Trường chính	Vai trò
Sidebar cấu hình	model_metadata	model_name, model_version, threshold	Hiển thị thông tin quản trị mô hình đang triển khai
Dải KPI đầu trang	dashboard_summary	total_transactions, flagged_transactions, fraud_ratio, pending_reviews	Cho biết trạng thái tổng quan của hệ thống
Bảng giao dịch 15 giây gần nhất	simulated_live_transactions	id, date, amount, merchant_id, fraud_label, ingest_time	Quan sát batch giao dịch mới nhất theo chu kỳ cập nhật
Danh sách cảnh báo tích lũy	realtime_alerts	fraud_score, alert_level, prediction_time, status_review	Theo dõi tập cảnh báo vượt ngưỡng
Tra cứu chi tiết	model_predictions	id, fraud_score, predicted_label, prediction_time, model_version	Tìm kiếm lịch sử dự đoán theo mã giao dịch
Duyệt giao dịch rủi ro	realtime_alerts	status_review, review_time	Cập nhật trạng thái xử lý thủ công
Biểu đồ phân tích	realtime_alerts + dashboard_summary	amount, prediction_time, top_risky_merchants	Phân tích phân bố rủi ro theo thời gian và theo merchant

Việc tách riêng collection cho từng chức năng giao diện giúp giảm độ phức tạp truy vấn. Dashboard không phải join động nhiều nguồn dữ liệu trong lúc hiển thị; thay vào đó, nó chỉ đọc đúng collection phù hợp với ngữ cảnh của từng thành phần. Đây là một điểm quan trọng để hệ thống duy trì được độ trễ thấp khi làm mới định kỳ.

Ba quyết định thiết kế cốt lõi. Thứ nhất, giao diện được đặt ở chế độ wide để ưu tiên vùng quan sát bảng dữ liệu. Thứ hai, kết nối MongoDB được cache theo resource nhằm tránh mở lại kết nối ở mỗi chu kỳ refresh. Thứ ba, lệnh `st_autorefresh` với chu kỳ 15.000 mili giây là nền tảng của cơ chế cập nhật gần thời gian thực.

3.4.4. Kiến trúc giao diện và nguyên tắc thiết kế trực quan

Mã nguồn dashboard sử dụng Streamlit kết hợp với CSS tùy biến để tạo giao diện dark mode. Đây là lựa chọn phù hợp cho một dashboard giám sát vì nền tối giúp làm nổi bật các điểm nhấn màu như chỉ báo KPI, thanh tiến trình fraud score và heatmap trên cột điểm rủi ro. Trong mã nguồn, các khối metric được bọc bởi nền xám đậm, header chính dùng dải chuyển sắc nhẹ, còn section-header dùng màu xanh lam để tạo tầng bậc thị giác rõ ràng.

Bố cục giao diện được chia thành bốn lớp. Lớp thứ nhất là header nhận diện hệ thống, nơi hiển thị tên dashboard và chu kỳ cập nhật. Lớp thứ hai là sidebar phục vụ cấu hình thời gian và xem metadata của mô hình. Lớp thứ ba là dải KPI tóm tắt tình hình hệ thống. Lớp thứ tư là vùng làm việc chính, trong đó người dùng chuyển giữa các tab nghiệp vụ rồi xuống phân biểu đồ phân tích xu hướng.

Điểm mạnh của bố cục này là duy trì được tính tuần tự trong trải nghiệm người dùng: từ nhận biết bối cảnh hệ thống, sang xem chỉ số tức thời, sang quan sát dữ liệu mới, sau đó mới đi vào tìm kiếm hoặc duyệt thủ công. Đây là cấu trúc hợp lý đối với một dashboard fraud detection, nơi người sử dụng cần đi từ tổng quan tới can thiệp cụ thể.

Cách định nghĩa KPI và tab trong Đoạn mã 3.4.2 phản ánh tư duy thiết kế dựa trên vai trò vận hành. Bốn KPI được chọn không ngẫu nhiên mà tương ứng với bốn câu hỏi quan trọng của bộ phận giám sát: hệ thống đã xử lý bao nhiêu giao dịch, hiện có bao nhiêu giao dịch bị gán cờ, mức rủi ro tổng thể là bao nhiêu và còn bao nhiêu trường hợp chưa được xử lý. Ba tab phía dưới lần lượt tương ứng với ba thao tác nghiệp vụ: quan sát, tra cứu và ra quyết định.

3.4.5. Mô tả chi tiết các phân hệ giao diện

3.4.5.1. Tab Giám sát Real-time

Tab Giám sát Real-time là vùng hiển thị dữ liệu vận hành tức thời của hệ thống. Thành phần đầu tiên trong tab này là bảng các giao dịch mới nhất trong chu kỳ 15 giây. Bảng này ưu tiên các trường ngắn nhưng có ý nghĩa nhận diện nhanh, bao gồm mã giao dịch, thời gian giao dịch, số tiền, merchant_id và fraud_label.

Thành phần tiếp theo là bảng cảnh báo tích lũy. Bảng này đọc trực tiếp từ collection realtime_alerts và sắp xếp theo prediction_time giảm dần. Cột fraud_score được tô nền dạng gradient đỏ để người dùng có thể nhận ra giao dịch nguy cơ cao chỉ bằng quan sát

nhANH, trong khi `alert_level` và `status_review` cung cấp ngữ nghĩa vận hành của từng cảnh báo.

Những ảnh chụp dashboard do tác giả cung cấp cho thấy bảng cảnh báo tích lũy đã phản ánh đúng logic trên: các bản ghi đều có `predicted_label = 1`, mức cảnh báo được tách thành Cao và Trung bình, còn cột `status_review` mặc định ở trạng thái Mới phát hiện trước khi chuyển sang khâu rà soát. Đây là bằng chứng cho thấy pipeline mô phỏng đã ghi dữ liệu cảnh báo vào MongoDB thành công và dashboard đọc lại đúng collection cần thiết.

3.4.5.2. Tab Tra cứu Chi tiết

Tab Tra cứu Chi tiết phục vụ nhu cầu kiểm tra lại lịch sử dự đoán theo một mã giao dịch cụ thể. Về mặt vận hành, đây là chức năng cần thiết vì một dashboard cảnh báo chỉ hữu ích khi người dùng có thể truy ngược từ tín hiệu bất thường về hồ sơ dự đoán tương ứng.

Trong mã nguồn, tab này đọc dữ liệu từ `model_predictions` rồi cung cấp hộp nhập ký tự để lọc theo trường `id`. Cách làm này giúp giảm số thao tác của người sử dụng: không cần viết truy vấn MongoDB, chỉ cần nhập chuỗi định danh giao dịch.

Ảnh chụp giao diện cho thấy tab Tra cứu Chi tiết hiển thị đúng các trường phục vụ kiểm tra mô hình như `fraud_score`, `predicted_label`, `prediction_time`, `model_name` và `model_version`. Như vậy, phần tra cứu không chỉ hỗ trợ nghiệp vụ mà còn hỗ trợ quản trị mô hình, vì người dùng có thể biết một dự đoán cụ thể được sinh ra bởi phiên bản mô hình nào.

3.4.5.3. Tab Duyệt Giao dịch Rủi ro

Đây là phân hệ quan trọng nhất về mặt vận hành, vì nó chuyển dashboard từ trạng thái quan sát sang trạng thái tương tác. Tab này chỉ lấy các bản ghi trong `realtime_alerts` có `status_review` thuộc nhóm Mới phát hiện hoặc Đang rà soát, nghĩa là chỉ hiện các trường hợp còn cần hành động.

Giao diện duyệt sử dụng `st.data_editor`, cho phép chỉnh sửa trực tiếp cột trạng thái xử lý trong khi vẫn giữ các cột khác ở dạng chỉ đọc. Các trường `amount`, `date` và `prediction_time` được định dạng phù hợp với ngữ cảnh nghiệp vụ, còn `fraud_score` được biểu diễn bằng progress bar để hỗ trợ ra quyết định nhanh.

Ảnh chụp màn hình cho thấy danh sách lựa chọn của status_review gồm bốn mức: Mới phát hiện, Đang rà soát, Đã xác nhận và Đã bỏ qua. Sự hiện diện của nút Lưu thay đổi đã duyệt cho thấy trạng thái trên dashboard không phải dữ liệu tạm thời trong phiên làm việc, mà được ghi ngược trở lại MongoDB. Điểm này rất quan trọng vì nó biến dashboard thành một công cụ bán tác nghiệp, chứ không chỉ là một lớp trình chiếu.

3.4.6. Lớp biểu đồ phân tích và giá trị diễn giải

Sau ba tab nghiệp vụ, dashboard tiếp tục cung cấp một lớp biểu đồ phân tích xu hướng. Về mặt nhận thức, đây là phần nối giữa dữ liệu vi mô của từng giao dịch với các mẫu rủi ro ở mức vĩ mô hơn. Người vận hành không chỉ thấy từng cảnh báo rời rạc mà còn có thể nhận biết các mẫu lặp lại.

Biểu đồ thứ nhất là xu hướng số lượng cảnh báo theo giờ hoặc theo ngày, được chọn thông qua radio button. Cấu trúc này cho phép dashboard phục vụ hai nhịp quan sát khác nhau: quan sát vi mô trong khung ngắn và quan sát xu hướng tích lũy theo chu kỳ dài hơn.

Biểu đồ thứ hai là histogram phân phối số tiền của các giao dịch nguy cơ gian lận. Ảnh chụp kết quả cho thấy phân phối có đuôi phải rõ rệt, số lượng lớn giao dịch tập trung ở vùng trung bình nhưng vẫn xuất hiện một số điểm rất cao. Điều này phù hợp với bản chất fraud detection, khi giá trị giao dịch lớn bất thường thường là tín hiệu quan trọng.

Biểu đồ thứ ba là bar chart Top Merchant có rủi ro cao. Về bản chất, biểu đồ này không dùng để kết luận ngay merchant nào là gian lận, mà để hỗ trợ định hướng kiểm tra: merchant xuất hiện lặp lại trong nhóm cảnh báo cần được ưu tiên phân tích thêm về ngữ cảnh giao dịch, loại hình kinh doanh và lịch sử hoạt động.



The screenshot shows a dashboard titled "Duyệt Giao dịch Rủi ro Cao" (Review High-Risk Transactions). It features a table with 7 columns: Id, Thời gian GD (Transaction Time), Số tiền (Amount), Điểm Fraud (Fraud Score), alert_level, Thời gian Dự đoán (Prediction Time), and Trạng thái Duyệt (Review Status). The table contains 12 rows of transaction data. The Fraud Score is visualized as a progress bar, and the alert_level is categorized as "Trung bình" (Average) or "Cao" (High). The Review Status includes options like "Mới phát hiện" (Newly discovered), "Đang rà soát" (Under review), and "Đã xác nhận" (Confirmed).

Id	Thời gian GD	Số tiền	Điểm Fraud	alert_level	Thời gian Dự đoán	Trạng thái Duyệt
SIM_4d3160d065754202	2026-04-16 06:19:42	\$1442.71	0.67	Trung bình	2026-04-16 06:30:40	Mới phát hiện
SIM_102c036aadf84a20	2026-04-16 06:19:42	\$335.81	0.68	Trung bình	2026-04-16 06:30:40	Mới phát hiện
SIM_1d754a5c3e8a498e	2026-04-16 06:19:42	\$1059.65	0.99	Cao	2026-04-16 06:29:59	Đang rà soát
SIM_719d41d73856432a	2026-04-16 06:19:42	\$1385.67	0.83	Trung bình	2026-04-16 06:29:38	Đã xác nhận
SIM_01a87309634c4722	2026-04-16 06:19:42	\$273.14	0.99	Cao	2026-04-16 06:29:17	Đã bỏ qua
SIM_cda3d0f25a2b4c6b	2026-04-16 06:19:42	\$983.16	0.62	Trung bình	2026-04-16 06:28:57	Mới phát hiện
SIM_10d458c93ffe49c3	2026-04-16 06:19:42	\$581.68	0.63	Trung bình	2026-04-16 06:28:57	Mới phát hiện
SIM_c1f20d9f2a8943cf	2026-04-16 06:19:42	\$826.41	0.68	Trung bình	2026-04-16 06:28:16	Mới phát hiện
SIM_4d1bedd3db434393	2026-04-16 06:19:42	\$1191.47	0.60	Trung bình	2026-04-16 06:28:16	Mới phát hiện
SIM_450a931d1a6843da	2026-04-16 06:19:42	\$1918.42	0.62	Trung bình	2026-04-16 06:27:15	Mới phát hiện

Hình 3. 3: Ảnh chụp kết quả thực nghiệm: tab Duyệt Giao dịch Rủi ro trên dashboard

Hình ở trên là một ảnh chụp trực tiếp từ dashboard thực nghiệm tại tab Duyệt Giao dịch Rủi ro. Hình cho thấy dữ liệu cảnh báo đã được đưa vào dạng bảng tương tác, trong đó fraud_score được hiển thị bằng progress bar, trạng thái status_review được chọn qua hộp selectbox và người dùng có thể lưu lại các thay đổi sau rà soát. Điều này xác nhận rằng dashboard không chỉ đọc dữ liệu từ MongoDB mà còn hỗ trợ thao tác cập nhật trạng thái xử lý theo đúng logic vận hành đã thiết kế.

3.4.7. Cơ chế mô phỏng gần thời gian thực và đồng bộ dữ liệu

Trong phạm vi đề án, hệ thống không sử dụng một nền tảng streaming công nghiệp như Kafka. Tuy nhiên, mã nguồn đã xây dựng một cơ chế micro-batch đủ để mô phỏng gần thời gian thực trong môi trường Google Colab. Các tham số cốt lõi gồm kích thước batch, chu kỳ nạp và ngưỡng cảnh báo được khai báo tách biệt, giúp tác giả dễ hiệu chỉnh theo mục đích trình diễn.

Luồng xử lý diễn ra theo thứ tự sau: lấy một lô giao dịch giả định từ synthetic_stream_df; tính fraud_score dựa trên amount, quantile lịch sử và nhãn fraud quá khứ; tạo predicted_label và alert_level; ghi đồng thời vào simulated_live_transactions, model_predictions và realtime_alerts; sau đó cập nhật lại dashboard_summary. Cách tổ chức này bảo đảm rằng mỗi chu kỳ làm mới của Streamlit luôn đọc được một ảnh chụp dữ liệu nhất quán.

Một điểm đáng chú ý trong ảnh chụp KPI là tổng số giao dịch lịch sử tăng từ 3.500 lên 3.700 giữa hai lần chụp, trong khi số giao dịch chờ rà soát vẫn được bảo toàn ở mức 31.

Điều đó cho thấy cơ chế tự làm mới của dashboard hoạt động đúng mục tiêu: dữ liệu mới được nạp thêm theo chu kỳ, nhưng trạng thái rà soát không bị mất do đã được lưu bền vững trong MongoDB.

Chu trình cập nhật gần thời gian thực của dashboard



Hình 3. 4: Chu trình cập nhật gần thời gian thực của dashboard

Tóm tắt chu trình cập nhật từ nguồn giao dịch mô phỏng đến lớp công bố giao diện. MongoDB là điểm hội tụ của dữ liệu vận hành, Streamlit là lớp đọc và hiển thị, còn ngrok chỉ đóng vai trò tạo đường dẫn public phục vụ demo.

3.4.8. Công bố dashboard qua ngrok

Sau khi Streamlit app được khởi động trên cổng 8501, ngrok được dùng để công bố giao diện ra Internet nhằm phục vụ trình bày đồ án. Điểm cần nhấn mạnh về mặt kiến trúc là ngrok không tham gia vào xử lý dữ liệu, không sinh cảnh báo và không lưu trạng thái nghiệp vụ. Nó chỉ mở một đường hầm HTTP an toàn từ môi trường chạy cục bộ đến một URL public.

Việc tách biệt rõ vai trò này giúp tránh nhầm lẫn giữa lớp hạ tầng phục vụ demo với lớp hạ tầng xử lý dữ liệu. Nói cách khác, nếu bỏ ngrok đi, dashboard vẫn hoạt động bình thường ở môi trường cục bộ; chỉ có khả năng chia sẻ ra bên ngoài bị mất.

Trong ngữ cảnh học thuật, lựa chọn ngrok là hợp lý vì nó giảm đáng kể chi phí triển khai, đồng thời cho phép đánh giá được trải nghiệm người dùng cuối trên một giao diện thật thay vì chỉ thông qua ảnh chụp hoặc notebook.

3.5. Kết luận chương 3

Chương 3 đã trình bày thiết kế tổng thể của hệ thống dữ liệu lớn cho bài toán phát hiện gian lận theo một kiến trúc logic dạng Lambda, trong đó dữ liệu lịch sử được xử lý theo batch path, còn dữ liệu mới hoặc dữ liệu mô phỏng gần thời gian thực được xử lý theo speed path. Hai luồng này hội tụ tại lớp serving layer, từ đó cung cấp dữ liệu cho dashboard giám sát và hỗ trợ quy trình rà soát nghiệp vụ. Cách tổ chức này cho thấy hệ thống không chỉ dừng ở mức xây dựng mô hình, mà đã được mở rộng thành một quy trình có khả năng lưu trữ, dự đoán, cảnh báo và trực quan hóa kết quả một cách nhất quán.

Về mặt lưu trữ, chương này đã biện luận và lựa chọn MongoDB Atlas làm kho dữ liệu trung tâm với mô hình document phù hợp cho dữ liệu giao dịch, dữ liệu đặc trưng, lịch sử dự đoán, cảnh báo thời gian thực, chỉ số tổng hợp và metadata của mô hình. Thiết kế schema và chỉ mục được xây dựng theo hướng phục vụ trực tiếp cho truy vấn dashboard, giảm độ phức tạp của lớp trình bày và tạo nền tảng cho việc mở rộng trong tương lai. Bên cạnh đó, phần tính toán dung lượng lưu trữ theo các kịch bản khác nhau, bao gồm cả trường hợp có replication factor, cho thấy hệ thống ở quy mô đồ án vẫn nằm trong ngưỡng triển khai khả thi trên MongoDB Atlas mà chưa cần đến kiến trúc phân tán sâu hơn.

Về mặt vận hành, dashboard Streamlit được xây dựng như lớp giao diện cuối cùng của hệ thống, cho phép hiển thị KPI, giao dịch mới, cảnh báo tích lũy, tra cứu chi tiết, duyệt giao dịch rủi ro và công bố demo qua ngrok. Điều này chứng minh rằng kết quả xử lý dữ liệu và mô hình hóa có thể được chuyển thành một công cụ giám sát có ý nghĩa thực tế. Như vậy, Chương 3 đã hoàn thành vai trò cầu nối giữa dữ liệu đã được chuẩn bị ở Chương 2 và thực nghiệm mô hình ở Chương 4, đồng thời khẳng định rằng đề tài không chỉ có giá trị ở cấp độ phân tích học thuật mà còn có khả năng tổ chức thành một hệ thống hỗ trợ ra quyết định trong bối cảnh phát hiện gian lận.

CHƯƠNG 4. THỰC NGHIỆM KHAI PHÁ VÀ KẾT QUẢ

4.1. Xây dựng mô hình

Sau khi hoàn tất tiền xử lý và kỹ thuật đặc trưng, dữ liệu được đưa vào giai đoạn thực nghiệm khai phá. Điểm quan trọng của Chương 4 là không đánh giá mô hình theo cách tách rời kỹ thuật với nghiệp vụ. Toàn bộ quy trình được tổ chức theo hướng: huấn luyện mô hình trên dữ liệu đã được chuẩn hóa; lựa chọn mô hình mạnh nhất theo Recall và các chỉ số hỗ trợ; hiệu chỉnh ngưỡng quyết định; chuyển đầu ra mô hình thành fraud score, predicted label và alert level; cuối cùng diễn giải kết quả theo tác động kinh tế.

4.1.1. Dữ liệu đầu vào cho mô hình

Đầu vào mô hình là file master_features.csv đã được tạo ở bước 2.3. Tập dữ liệu này gồm 8.914.963 dòng, trong đó ngoài các biến gốc còn có các biến dẫn xuất như hour, day_of_week, is_weekend, is_night, amount_log, amt_night_interaction, client_avg_amt, amt_diff_from_client_avg và client_tx_count. Sau khi loại các trường không dùng trực tiếp để huấn luyện như id, date và fraud_label, tập test dùng cho đánh giá cuối cùng có 1.782.993 bản ghi với 26 cột đặc trưng; tỷ lệ fraud ở tập test vẫn là 0,001495, cho thấy chiến lược stratified split đã giữ được phân bố lớp gốc.

Bảng 4. 1: Tổng quan dữ liệu đánh giá mô hình

Chỉ tiêu	Giá trị
Loại mô hình tốt nhất đã nạp	XGBClassifier
Số dòng tập test	1.782.993
Số cột đặc trưng ở tập test	26
Tỷ lệ fraud ở tập test	0,001495

4.1.2. Các mô hình được thực nghiệm

Đề tài so sánh ba mô hình: Logistic Regression, Random Forest và XGBoost. Logistic Regression được xem là mô hình baseline do đơn giản, dễ diễn giải và thường được dùng làm mốc so sánh. Random Forest đại diện cho nhóm ensemble theo bagging, có khả năng học quan hệ phi tuyến nhưng tương đối ổn định. XGBoost đại diện cho nhóm boosting, vốn nổi tiếng với hiệu quả cao trên dữ liệu bảng và khả năng tối ưu tốt trong các bài toán phân lớp mất cân bằng.

Việc chỉ chọn ba mô hình không làm giảm giá trị của thực nghiệm; ngược lại, nó giúp quy trình đánh giá tập trung và nhất quán. Trong bối cảnh đồ án, mục tiêu không phải là quét hết mọi thuật toán có thể, mà là chọn một tập mô hình đủ đại diện để rút ra kết luận về hướng tiếp cận phù hợp cho fraud detection.

4.1.3. Tinh chỉnh siêu tham số bằng GridSearchCV

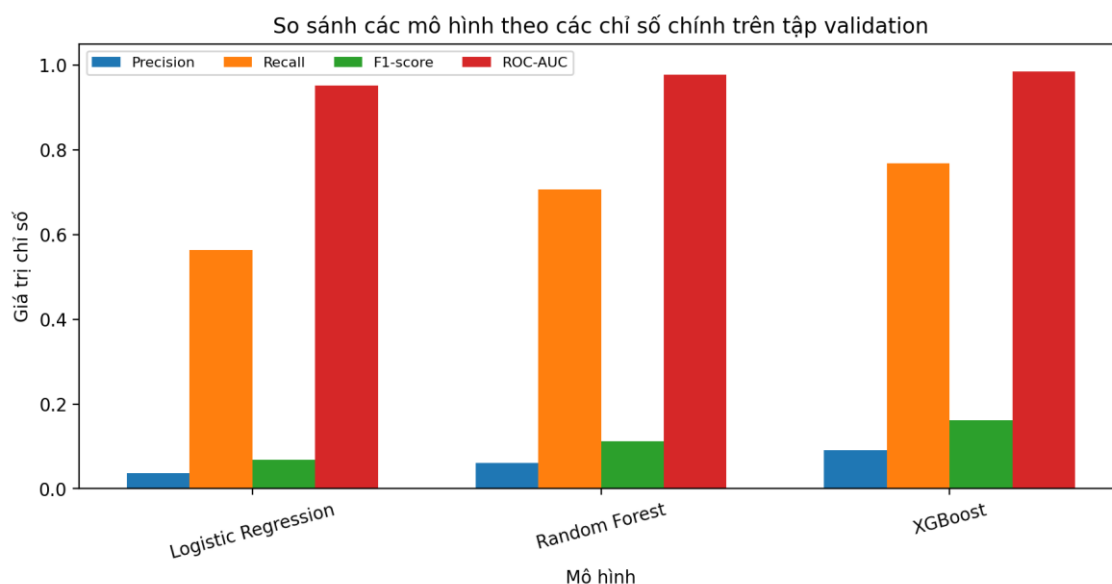
Quy trình huấn luyện sử dụng GridSearchCV với scoring = 'recall'. Lựa chọn này phản ánh đúng trọng tâm của bài toán: ưu tiên bắt được nhiều fraud hơn là tối đa hóa Accuracy. Với Logistic Regression, không gian tìm kiếm tham số được giữ gọn để tránh chi phí tính toán quá lớn trong môi trường Colab. Với Random Forest và XGBoost, các tham số cốt lõi như số cây, độ sâu, learning rate và subsample được khai báo theo grid rút gọn nhưng vẫn đủ để so sánh.

Sau mỗi lần huấn luyện, mô hình tốt nhất và training result tương ứng được lưu ra file joblib để phục vụ tái sử dụng trong giai đoạn đánh giá kỹ thuật. Cách tổ chức này giúp quy trình thực nghiệm có khả năng lặp lại, đồng thời giảm áp lực bộ nhớ do không phải giữ toàn bộ mô hình trong RAM.

4.1.4. Kết quả so sánh mô hình trên validation

Bảng 4. 2: So sánh kết quả huấn luyện trên tập validation

Mô hình	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0,9771	0,0365	0,5632	0,0685	0,9520
Random Forest	0,9833	0,0609	0,7068	0,1121	0,9774
XGBoost	0,9881	0,0906	0,7675	0,1620	0,9850



Hình 4. 1: So sánh ba mô hình trên tập validation

Kết quả trên cho thấy XGBoost là mô hình mạnh nhất trong ba mô hình đã thử nghiệm. XGBoost không chỉ có Recall cao nhất (0,7675) mà còn đồng thời đạt Precision, F1-score và ROC-AUC cao nhất. Điều này cho thấy mô hình vừa bắt được nhiều fraud hơn, vừa có năng lực phân biệt hai lớp tốt hơn trên dữ liệu validation. Random Forest đứng thứ hai, còn Logistic Regression đóng vai trò baseline hợp lý nhưng kém hơn rõ rệt về Recall và F1-score.

Việc chọn XGBoost là mô hình triển khai không dựa trên cảm tính mà dựa trên logic nhất quán với bài toán. Khi mục tiêu ưu tiên là phát hiện gian lận, mô hình có Recall cao hơn và ROC-AUC cao hơn sẽ có giá trị hơn cho bước threshold tuning và đánh giá vận hành sau đó.

4.2. Đánh giá kỹ thuật

4.2.1. Vì sao không dùng Accuracy làm chỉ số chính

Dữ liệu của đề tài cực mất cân bằng, với tỷ lệ fraud chỉ khoảng 0,1495%. Trong bối cảnh này, Accuracy cao không có nghĩa là mô hình hữu ích. Một mô hình có thể dự đoán gần như tất cả giao dịch là bình thường và vẫn đạt Accuracy rất cao, nhưng lại bỏ sót phần lớn fraud. Do đó, đề tài chỉ xem Accuracy như chỉ số tham khảo, còn các chỉ số trung tâm là Recall, Precision, F1-score và ROC-AUC. Đây là quyết định hoàn toàn phù hợp với cơ sở lý thuyết của fraud detection và với chính bản chất dữ liệu thực nghiệm.

4.2.2. Đánh giá tại ngưỡng mặc định 0,5

Bảng 4. 3: Kết quả XGBoost trên tập test tại threshold = 0,5

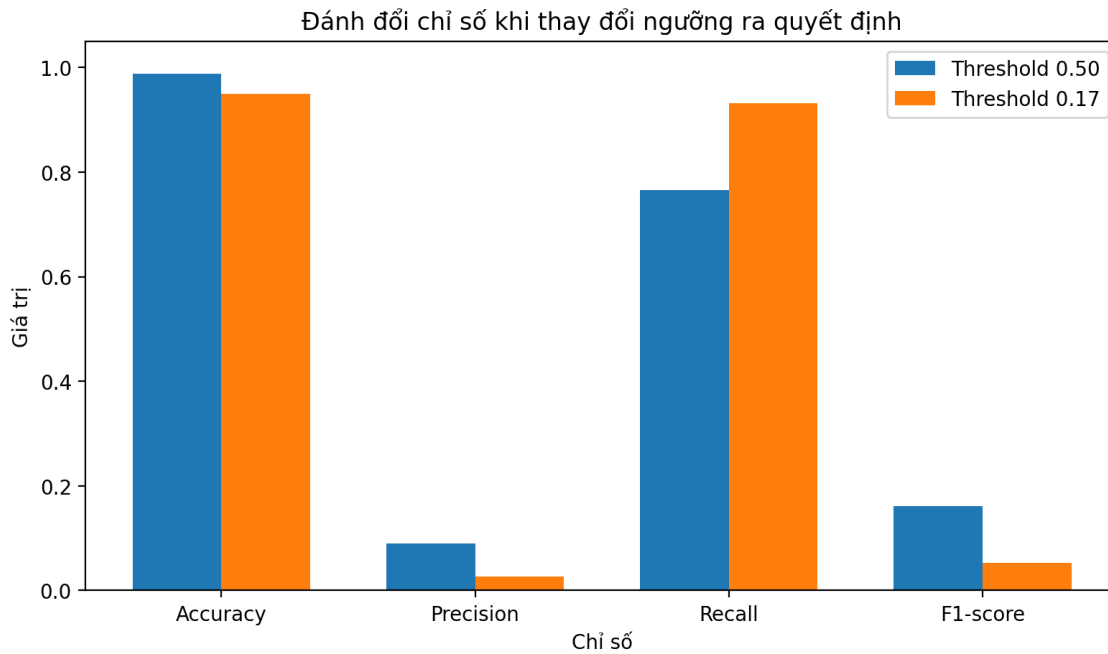
Chỉ số	Giá trị
Accuracy	0,988035
Precision	0,089718
Recall	0,765566
F1-score	0,160614
ROC-AUC	0,986074
TN	1.759.619
FP	20.708
FN	625
TP	2.041

Ở threshold mặc định 0,5, XGBoost cho kết quả tương đối tốt về mặt kỹ thuật. ROC-AUC = 0,986074 cho thấy mô hình có khả năng phân biệt hai lớp rất mạnh. Recall = 0,765566 nghĩa là mô hình phát hiện được khoảng 76,56% số giao dịch gian lận thật. Tuy nhiên, 625 fraud vẫn bị bỏ sót. Precision = 0,089718 cũng còn thấp, nghĩa là trong số các giao dịch bị gán cờ fraud, chỉ khoảng 8,97% là fraud thật.

Nếu nhìn đơn thuần vào Accuracy = 0,988035, ta có thể nhầm tưởng mô hình đã đủ tốt. Nhưng dưới góc nhìn nghiệp vụ, bỏ sót 625 fraud là một chi phí đáng kể. Đây là lý do threshold tuning trở thành bước bắt buộc: mục tiêu của hệ thống không phải giữ Accuracy thật cao mà là giảm FN xuống mức thấp hơn.

4.2.3. Threshold tuning và lựa chọn ngưỡng vận hành

Threshold tuning được thực hiện bằng cách quét các ngưỡng xác suất dự đoán, sau đó đánh giá lại theo Recall, Precision, F1-score và bối cảnh vận hành. Tiêu chí lựa chọn không đơn thuần là chọn ngưỡng có F1 lớn nhất, mà là chọn ngưỡng giúp Recall đạt mức cao trong khi khối lượng cảnh báo vẫn còn có thể quản lý được. Trên dữ liệu của đề tài, ngưỡng 0,17 được lựa chọn là ngưỡng vận hành phù hợp hơn so với 0,5.



Hình 4. 2: So sánh hiệu quả giữa threshold mặc định và threshold vận hành

4.2.4. Kết quả cuối sau khi chọn threshold = 0,17

Bảng 4. 4: Kết quả cuối của XGBoost tại threshold = 0,17

Chỉ số	Giá trị
Accuracy	0,950046
Precision	0,027197
Recall	0,932108
F1-score	0,052851
ROC-AUC	0,986074
Inference time (sec)	2,887183
Avg inference (ms/record)	0,001619
TN	1.691.440
FP	88.887
FN	181
TP	2.485

Khi hạ threshold từ 0,5 xuống 0,17, Recall tăng mạnh từ 0,765566 lên 0,932108; tương ứng, số fraud bị bỏ sót giảm từ 625 xuống còn 181. Đây là cải thiện rất quan trọng về mặt vận hành, vì giảm FN luôn là mục tiêu ưu tiên hàng đầu trong fraud detection. Đồng thời, ROC-AUC vẫn giữ ở mức 0,986074, cho thấy năng lực phân biệt lớp của mô hình không thay đổi.

Tuy nhiên, cái giá phải trả là Precision giảm xuống 0,027197 và số FP tăng lên 88.887. Điều này có nghĩa là hệ thống bắt được nhiều fraud hơn nhưng phải chấp nhận khối lượng cảnh báo sai lớn hơn. Đây là một đánh đổi hợp lý trong bối cảnh đề tài ưu tiên giảm bỏ sót

fraud hơn là giảm chi phí cảnh báo sai ngay từ đầu. Về tốc độ, mô hình chỉ mất 2,887183 giây để suy luận trên toàn bộ tập test 1.782.993 bản ghi, tương đương 0,001619 ms/bản ghi, cho thấy nó đủ nhanh để xem xét tích hợp vào pipeline batch hoặc micro-batch.

4.3. Phân tích tác động kinh tế và khả năng triển khai

4.3.1. Chuyển từ đánh giá kỹ thuật sang đánh giá nghiệp vụ

Việc đánh giá mô hình trong đề tài này không dừng ở các chỉ số kỹ thuật như Recall, Precision hay ROC-AUC, mà cần được mở rộng sang góc nhìn nghiệp vụ. Một mô hình có kết quả kỹ thuật tốt chưa chắc đã mang lại hiệu quả vận hành nếu không giúp giảm tổn thất do gian lận hoặc làm phát sinh quá nhiều cảnh báo sai. Vì vậy, kết quả dự báo cần được diễn giải lại theo giá trị sử dụng thực tế của hệ thống.

Trong cách tiếp cận này, confusion matrix được xem như một khung chi phí – lợi ích. TP là các giao dịch gian lận được phát hiện đúng, nhờ đó có thể hỗ trợ ngăn chặn hoặc giảm thiệt hại tài chính. FN là các giao dịch gian lận bị bỏ sót, gây tổn thất trực tiếp vì fraud vẫn đi qua hệ thống mà không bị phát hiện. FP là các giao dịch hợp lệ nhưng bị cảnh báo nhầm, làm tăng chi phí rà soát thủ công và có thể ảnh hưởng đến trải nghiệm khách hàng. Trong khi đó, TN là các giao dịch bình thường được nhận diện đúng và không cần xử lý thêm.

Cách diễn giải này giúp chuyển kết quả mô hình từ mức “độ chính xác kỹ thuật” sang mức “tác động vận hành”. Nói cách khác, mô hình không chỉ được đánh giá bằng khả năng dự đoán đúng, mà còn bằng mức độ giảm tổn thất do fraud và khả năng kiểm soát chi phí phát sinh từ cảnh báo sai. Đây là cơ sở để tiếp tục phân tích hiệu quả kinh tế của hệ thống ở các mục sau.

4.3.2. Giả định chi phí – lợi ích

Do đề tài không có dữ liệu vận hành thực của một doanh nghiệp cụ thể, các tham số kinh tế được xây dựng dưới dạng giả định để minh họa phương pháp đánh giá. Cụ thể, chi phí cho mỗi FP được giả định bằng 5 đơn vị giá trị; chi phí cho mỗi FN là 250; lợi ích của mỗi TP là 300. Đây không phải các con số áp dụng chung cho mọi tổ chức, mà là bộ giả định để cho thấy cách một hệ thống fraud detection có thể được lượng hóa về mặt kinh tế.

Bảng 4. 5: Bộ giả định chi phí – lợi ích trong phân tích kinh tế

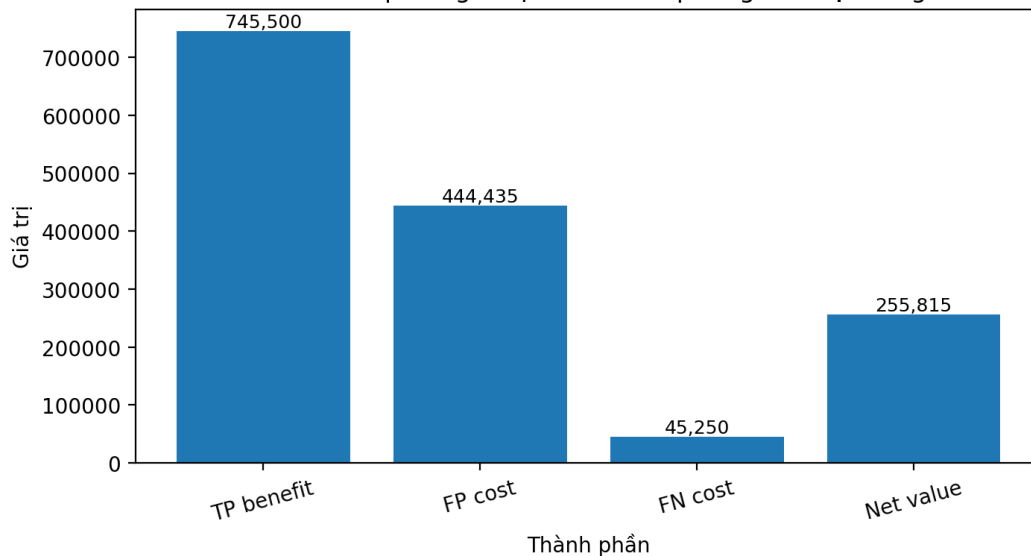
Thành phần	Giả định
Benefit cho mỗi TP	300
Cost cho mỗi FP	5
Cost cho mỗi FN	250

4.3.3. Giá trị kinh tế ròng của mô hình

Bảng 4. 6: Ước tính tác động kinh tế tại threshold = 0,17

Thành phần	Công thức	Giá trị
TP benefit	2.485×300	745.500
FP cost	88.887×5	444.435
FN cost	181×250	45.250
Net business value	TP benefit – FP cost – FN cost	255.815

Các thành phần giá trị kinh tế mô phỏng của hệ thống



Hình 4. 3: Cấu phần giá trị kinh tế ròng của mô hình tại threshold = 0,17

Kết quả cho thấy dù hệ thống tạo ra số lượng FP lớn, tổng lợi ích từ việc phát hiện đúng fraud vẫn lớn hơn tổng chi phí phát sinh từ cảnh báo sai và phần tổn thất còn lại do bỏ sót fraud. Giá trị kinh tế ròng theo bộ giả định là 255.815 đơn vị giá trị. Đây là một kết quả có ý nghĩa vì nó chứng minh rằng mô hình không chỉ tốt trên phương diện chỉ số kỹ thuật mà còn có thể tạo lợi ích ròng dương trong một kịch bản vận hành giả định.

Một điểm đáng chú ý là chi phí lớn nhất không còn đến từ FN mà đến từ FP. Điều này phản ánh đúng bản chất của threshold 0,17: mô hình ưu tiên bắt được nhiều fraud, chấp nhận mở rộng phạm vi cảnh báo. Từ góc nhìn cải tiến hệ thống, điều cần làm tiếp theo không chỉ là tiếp tục tăng Recall, mà là tìm cách giảm FP thông qua đặc trưng tốt hơn, threshold thông minh hơn hoặc phân tầng hành động theo mức rủi ro.

4.3.4. Đánh giá threshold dưới góc nhìn vận hành

Bảng 4. 7: Chỉ số vận hành của threshold = 0,17

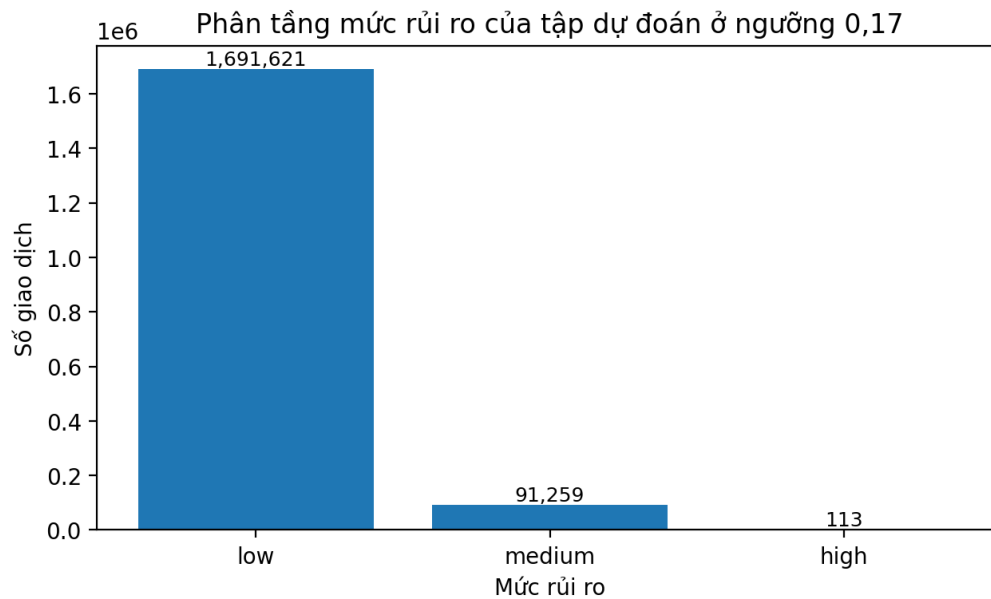
Chỉ số vận hành	Giá trị
Alert rate	0,051248
Recall	0,932108
Fraud miss rate	0,067892
Precision	0,027197
Số cảnh báo cần để tìm 1 fraud thật	36,769416
Số cảnh báo sai cho mỗi fraud thật	35,769416

Ở ngưỡng 0,17, hệ thống gắn cờ khoảng 5,12% tổng số giao dịch. Đây là tỷ lệ không quá nhỏ, nhưng vẫn nằm trong một mức có thể chấp nhận đối với bài toán cảnh báo. Quan trọng hơn, fraud miss rate chỉ còn 0,067892, tức khoảng 6,79% số fraud thật bị bỏ sót. Dưới góc nhìn hạn chế để lọt gian lận, đây là một kết quả tích cực.

Ngược lại, Precision thấp cho thấy chất lượng cảnh báo còn hạn chế nếu xét theo tiêu chí tự động hóa hoàn toàn. Hệ thống cần khoảng 36,77 cảnh báo để tìm được 1 fraud thật. Chính vì vậy, kết luận vận hành hợp lý là mô hình phù hợp nhất với vai trò alerting_and_manual_review, tức phát cảnh báo sớm và hỗ trợ bộ phận rà soát thủ công, thay vì dùng để tự động chặn giao dịch ở quy mô lớn.

4.3.5. Phân tầng rủi ro để phục vụ dashboard

Để hỗ trợ trực quan hóa trên dashboard, tập dữ liệu ở ngưỡng 0,17 được chia thành ba mức rủi ro: low, medium và high. Kết quả cho thấy phần lớn giao dịch nằm ở nhóm low với 1.691.621 giao dịch, trong khi nhóm medium có 91.259 giao dịch và nhóm high chỉ có 113 giao dịch. Cấu trúc này cho thấy sau khi áp dụng ngưỡng cảnh báo, hệ thống vẫn giữ được khả năng tách dữ liệu thành các mức ưu tiên khác nhau thay vì chỉ phân loại nhị phân đơn thuần.



Hình 4. 4: Phân bố giao dịch theo ba mức rủi ro trên dashboard

Về mặt vận hành, việc phân tầng này có ý nghĩa quan trọng đối với dashboard và quy trình rà soát. Nhóm high có quy mô rất nhỏ nên có thể được ưu tiên kiểm tra trước, phù hợp với vai trò cảnh báo mức nghiêm trọng. Nhóm medium đóng vai trò là lớp giao dịch cần theo dõi thêm hoặc đưa vào rà soát chọn lọc. Trong khi đó, nhóm low chiếm đa số và có thể được xem như lớp nền ít rủi ro hơn trong quá trình giám sát.

Như vậy, phân tầng rủi ro giúp dashboard không chỉ hiển thị giao dịch bị gấn cò, mà còn cung cấp một cơ chế ưu tiên xử lý theo mức độ nghi ngờ. Điều này làm tăng giá trị sử dụng của mô hình trong thực tế, vì bộ phận nghiệp vụ có thể tập trung nguồn lực vào các giao dịch có khả năng gian lận cao nhất thay vì rà soát dàn trải toàn bộ tập cảnh báo.

4.3.6. Cập nhật thiết kế về hiệu quả kinh tế

4.3.6.1. Truy xuất source code và logic dữ liệu

Phần dashboard hiệu quả kinh tế được xây dựng trên cơ sở mở rộng dashboard giám sát gian lận đã có ở Chương 3, thay vì thiết kế lại một giao diện độc lập. Về mặt dữ liệu, cấu trúc này được tổ chức theo hai lớp bổ sung. Lớp thứ nhất là lớp tổng hợp giá trị kinh tế, có nhiệm vụ đọc dữ liệu từ tập cảnh báo vận hành và chuyển trạng thái rà soát thành các chỉ số kinh tế có thể theo dõi. Lớp thứ hai là lớp trình bày trên dashboard, nơi các chỉ số kinh tế được đưa vào giao diện dưới dạng KPI, tab chức năng riêng và biểu đồ xu hướng

theo thời gian. Cách tổ chức này giúp phần kinh tế được tích hợp trực tiếp vào hệ thống hiện có, đồng thời vẫn giữ được tính nhất quán với các chỉ số giám sát truyền thống.

Nguồn dữ liệu trung tâm của phần mở rộng này là collection `realtime_alerts`, vì đây là nơi lưu toàn bộ các giao dịch đã bị hệ thống gắn cờ cùng với trạng thái xử lý nghiệp vụ. Từ collection này, hệ thống xác định số fraud đã được xác nhận, số cảnh báo bị loại bỏ và số trường hợp còn đang chờ rà soát. Trên cơ sở đó, một collection tổng hợp mới được tạo ra để lưu giá trị kinh tế hiện tại của hệ thống, phục vụ hiển thị nhanh trên dashboard. Song song với đó, một collection khác được dùng để lưu chuỗi thời gian của giá trị kinh tế, nhằm hỗ trợ trực quan hóa xu hướng thay đổi qua các lần cập nhật. Trong khi đó, collection `dashboard_summary` vẫn được giữ nguyên vai trò cung cấp các KPI giám sát cơ bản như tổng giao dịch, số cảnh báo và tỷ lệ rủi ro, tức là lớp kinh tế được bổ sung thêm chứ không thay thế phần dashboard trước đó.

Về logic xử lý, hệ thống thực hiện một bước tổng hợp định kỳ từ dữ liệu cảnh báo đã có trong MongoDB để tính ra giá trị kinh tế ròng và ghi kết quả vào hai collection chuyên biệt nói trên. Nhờ đó, dashboard có thể đồng thời hiển thị cả trạng thái giám sát fraud và tác động kinh tế của hệ thống trong cùng một giao diện. Cách triển khai này cho thấy phần đánh giá hiệu quả kinh tế không phải là một nội dung trình bày tách rời, mà được xây dựng trực tiếp từ dòng dữ liệu vận hành thực tế của dashboard, qua đó làm rõ hơn mối liên hệ giữa kết quả mô hình, quá trình rà soát và giá trị sử dụng của hệ thống trong bối cảnh nghiệp vụ.

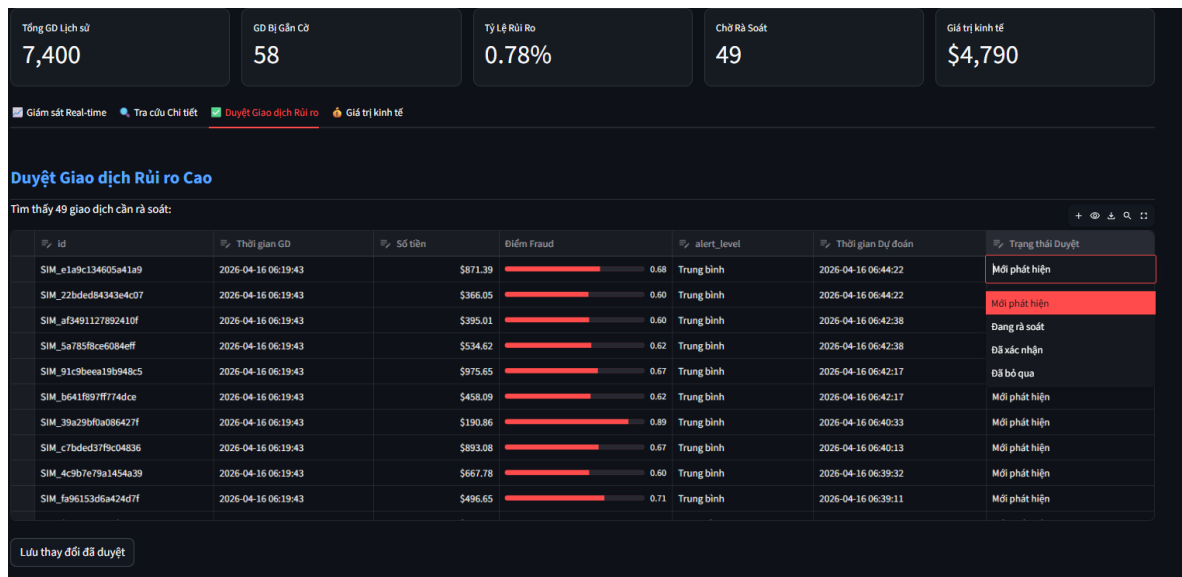
Về hàm `update_business_value_summary` được dùng để tổng hợp dữ liệu; khi vá vào dashboard, hàm này xuất hiện dưới tên `recompute_business_value_summary`. Hàm đọc cảnh báo từ `realtime_alerts`, tính giá trị ròng và ghi kết quả vào hai collection mới là `business_value_summary` và `business_value_snapshots`.

Công thức mô phỏng: $\text{total_business_value} = \text{confirmed_fraud} * X - \text{dismissed_alerts} * Y$

Trong đó, `confirmed_fraud` là số cảnh báo đã xác nhận là gian lận thật; `dismissed_alerts` là số cảnh báo đã bị bỏ qua. Hai hệ số `X` và `Y` là hai hệ số chỉ mang tính mô phỏng để minh họa nguyên tắc lượng hóa giá trị.

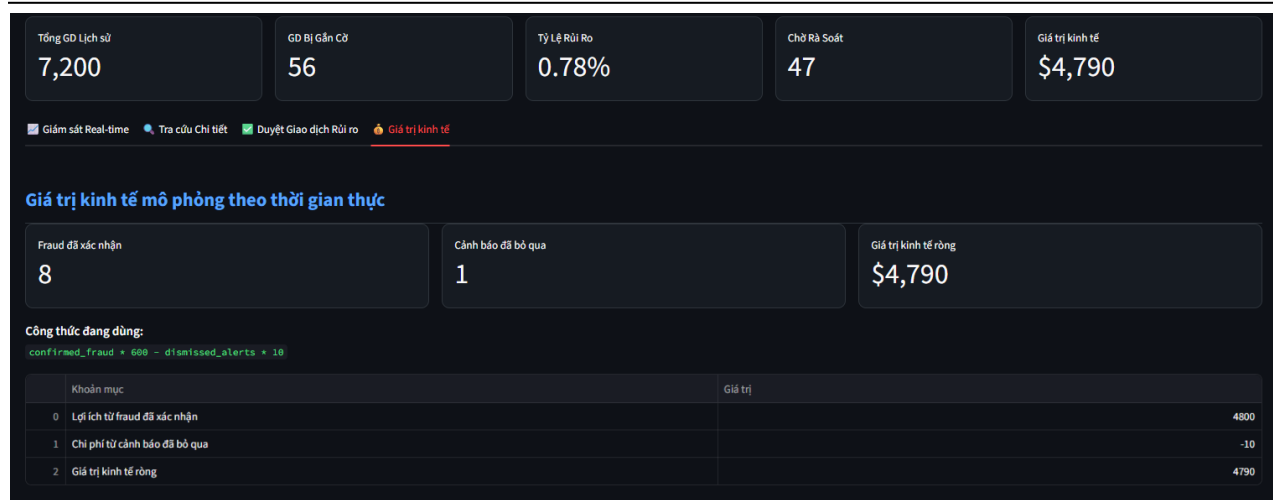
4.3.6.2. Thiết kế giao diện sau khi bổ sung

Sau khi được cập nhật, dashboard không còn chỉ đóng vai trò giám sát rủi ro, mà được mở rộng thành một giao diện kết hợp giữa theo dõi vận hành và đánh giá hiệu quả nghiệp vụ. Về cấu trúc, giao diện được tổ chức theo hai lớp thông tin. Lớp thứ nhất là lớp giám sát rủi ro với các KPI cơ bản như tổng số giao dịch, số giao dịch bị gán cờ, tỷ lệ rủi ro và số hồ sơ chờ rà soát. Lớp thứ hai là lớp hiệu quả kinh tế, được thể hiện bằng KPI Giá trị kinh tế và tab chức năng riêng, giúp người dùng quan sát trực tiếp giá trị mà hệ thống tạo ra từ kết quả phát hiện gian lận.



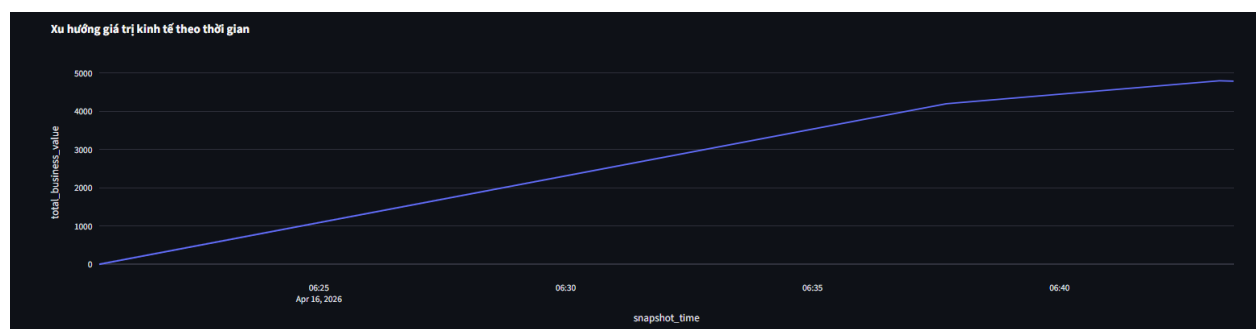
Hình 4. 5: Dashboard sau khi bổ sung KPI “Giá trị kinh tế” và liên kết với tab duyệt giao dịch rủi ro

Việc mở rộng phần đầu trang từ bốn lên năm KPI có ý nghĩa rõ ràng về mặt thiết kế. KPI Giá trị kinh tế được đặt cùng hàng với các chỉ số giám sát chính, cho thấy yếu tố kinh tế đã được tích hợp vào cùng một tầng quan sát với trạng thái vận hành của hệ thống. Nhờ đó, người dùng không chỉ biết hệ thống đang xử lý bao nhiêu giao dịch và tạo ra bao nhiêu cảnh báo, mà còn thấy ngay tác động nghiệp vụ tương ứng tại thời điểm theo dõi. Ở tab Duyệt Giao dịch Rủi ro, các cảnh báo tiếp tục được gắn với bốn trạng thái xử lý gồm Mới phát hiện, Đang rà soát, Đã xác nhận và Đã bỏ qua. Trường trạng thái này đóng vai trò kết nối giữa giao diện tác nghiệp và lớp tính toán hiệu quả kinh tế, vì kết quả rà soát sẽ quyết định giao dịch nào được ghi nhận là fraud thật và giao dịch nào được xem là cảnh báo không cần xử lý.



Hình 4. 6: Tab “Giá trị kinh tế” hiển thị KPI

Tab Giá trị kinh tế được thiết kế như một lớp diễn giải riêng cho phần tác động nghiệp vụ của hệ thống. Bên trong tab này, giao diện gồm ba thành phần chính: nhóm KPI tức thời, công thức tính đang sử dụng và bảng phân rã lợi ích – chi phí. Cách bố trí này giúp người dùng không chỉ nhìn thấy kết quả cuối cùng, mà còn hiểu được giá trị đó được hình thành từ đâu. Trong ví dụ minh họa, dashboard ghi nhận 8 fraud đã xác nhận và 1 cảnh báo đã bỏ qua, từ đó tính ra giá trị kinh tế ròng là 4.790 USD. Con số này được giải thích trực tiếp bằng bảng phân rã gồm 4.800 USD lợi ích từ fraud đã xác nhận đúng và 10 USD chi phí từ cảnh báo bị bỏ qua, qua đó tăng tính minh bạch cho phần đánh giá.



Hình 4. 7: Biểu đồ xu hướng giá trị kinh tế tích lũy theo thời gian dựa trên collection business_value_snapshots

Ngoài phần KPI và bảng tổng hợp, giao diện còn bổ sung biểu đồ xu hướng giá trị kinh tế theo thời gian. Biểu đồ này không nhằm phản ánh doanh thu kế toán, mà dùng để theo dõi quá trình tích lũy giá trị kinh tế mô phỏng khi trạng thái xử lý cảnh báo thay đổi qua các chu kỳ vận hành. Nhờ việc lưu lại các mốc giá trị theo thời gian, dashboard có thể cho thấy không chỉ “giá trị hiện tại là bao nhiêu” mà còn “giá trị đó đã tăng lên như thế nào”. Điều này làm cho giao diện trở nên hoàn chỉnh hơn: từ một công cụ giám sát fraud,

dashboard được mở rộng thành một công cụ vừa hỗ trợ rà soát rủi ro, vừa giúp diễn giải tác động kinh tế của hệ thống trong bối cảnh vận hành thực tế.

4.3.7. Lưu ý về business insight từ feature importance

Phân phân tích tác động kinh tế trong đề tài này cần được gắn với giá trị nghiệp vụ của mô hình. Tuy nhiên, bộ kết quả hiện có chưa xuất ra một bảng feature importance chuẩn hóa để làm căn cứ định lượng cho việc xếp hạng mức độ quan trọng của từng biến. Vì vậy, báo cáo không đưa ra các kết luận chi tiết theo kiểu “biến nào quan trọng nhất” nếu chưa có bằng chứng trực tiếp từ kết quả thực nghiệm, nhằm bảo đảm tính chính xác và chặt chẽ về mặt học thuật.

Thay cho cách diễn giải dựa trên một bảng xếp hạng biến chưa được xác nhận, các insight nghiệp vụ trong đề tài được rút ra từ những nguồn thông tin đã được kiểm chứng trong quá trình phân tích. Thứ nhất là kết quả EDA, cho thấy rủi ro gian lận thay đổi theo khung giờ và ngày trong tuần, từ đó khẳng định yếu tố thời gian có giá trị trong phát hiện fraud. Thứ hai là logic của các đặc trưng đã được xây dựng như `is_night`, `amt_night_interaction`, `client_avg_amt` và `amt_diff_from_client_avg`, vốn phản ánh hành vi bất thường theo thời gian và độ lệch so với lịch sử giao dịch. Thứ ba là phản ứng của hệ thống khi hạ threshold để ưu tiên bắt fraud, qua đó cho thấy mô hình phù hợp hơn với mục tiêu cảnh báo sớm và hỗ trợ rà soát thủ công.

4.4. Kết luận chương 4

Chương 4 đã trình bày toàn bộ quá trình thực nghiệm mô hình phát hiện gian lận trên tập dữ liệu đã được tiền xử lý và tạo đặc trưng ở các chương trước. Ba mô hình gồm Logistic Regression, Random Forest và XGBoost được huấn luyện và so sánh trên cùng một tập dữ liệu, trong đó XGBoost cho kết quả tốt nhất về Recall, F1-score và ROC-AUC. Kết quả này cho thấy XGBoost là lựa chọn phù hợp nhất cho bài toán fraud detection trong bối cảnh dữ liệu rất mất cân bằng, nơi mục tiêu quan trọng không phải là tối đa hóa Accuracy mà là phát hiện được nhiều giao dịch gian lận nhất có thể.

Trên cơ sở mô hình tốt nhất, chương này tiếp tục thực hiện threshold tuning và lựa chọn ngưỡng vận hành 0,17 thay cho ngưỡng mặc định 0,5. Việc hạ ngưỡng giúp Recall tăng từ 0,765566 lên 0,932108, đồng thời giảm số fraud bị bỏ sót từ 625 xuống còn 181.

Dù Precision giảm và số lượng cảnh báo sai tăng lên, kết quả này vẫn phù hợp với định hướng của đề tài là ưu tiên hạn chế để lọt fraud hơn là giảm cảnh báo ngay từ đầu. Từ góc nhìn vận hành, mô hình ở ngưỡng 0,17 phù hợp nhất với vai trò cảnh báo sớm và hỗ trợ rà soát thủ công, thay vì dùng để tự động từ chối giao dịch trên diện rộng.

Bên cạnh đánh giá kỹ thuật, Chương 4 còn chuyển kết quả mô hình sang góc nhìn nghiệp vụ và kinh tế. Confusion matrix được diễn giải lại thành khung chi phí – lợi ích; threshold được đánh giá dưới góc độ alert rate, fraud miss rate và chi phí rà soát; đồng thời dashboard được mở rộng để hiển thị KPI giá trị kinh tế, bảng phân rã lợi ích – chi phí và biểu đồ xu hướng giá trị theo thời gian. Các kết quả này cho thấy mô hình không chỉ có ý nghĩa về mặt dự đoán, mà còn có khả năng hỗ trợ ra quyết định trong vận hành thực tế. Như vậy, Chương 4 đã hoàn thành vai trò kiểm chứng giá trị của hệ thống ở cả ba mức: mức kỹ thuật, mức vận hành và mức hiệu quả kinh tế, tạo cơ sở trực tiếp cho phần kết luận và kiến nghị ở chương cuối.

CHƯƠNG 5. KẾT LUẬN VÀ KIẾN NGHỊ

5.1. Kết luận

Đề tài “Phát hiện gian lận trong thanh toán trực tuyến trên dữ liệu giao dịch lớn” đã được triển khai theo đúng logic mà hướng dẫn báo cáo yêu cầu: từ vấn đề kinh tế, qua thu thập và làm sạch dữ liệu, thiết kế hạ tầng lưu trữ, khai phá dữ liệu và cuối cùng là giải pháp kinh doanh. Trên nền tảng bộ dữ liệu Kaggle có quy mô lớn, đề tài đã xây dựng được một pipeline nhất quán bao gồm tích hợp dữ liệu, xử lý missing values, outliers, biến đổi và mã hóa, kỹ thuật đặc trưng, huấn luyện mô hình, threshold tuning và tổ chức đầu ra cho hệ thống giám sát.

Về mặt dữ liệu, đề tài đã tích hợp thành công 8.914.963 giao dịch có nhãn từ hai nguồn dữ liệu khác nhau, làm rõ bản chất cực mất cân bằng của bài toán với tỷ lệ fraud chỉ 0,1495%. Về mặt mô hình, XGBoost được lựa chọn là mô hình triển khai phù hợp nhất. Về mặt vận hành, threshold = 0,17 được xác định là ngưỡng cân bằng tốt hơn giữa yêu cầu bắt fraud và chi phí bỏ sót, cho dù kéo theo số lượng cảnh báo sai cao hơn. Về mặt hệ thống, MongoDB Atlas, Streamlit và ngrok đã được tổ chức thành một kiến trúc có khả năng lưu trữ, cảnh báo và hiển thị kết quả gần thời gian thực.

Một kết quả quan trọng của đề tài là đã chuyển hóa các chỉ số kỹ thuật thành ngôn ngữ kinh tế. Dưới bộ giả định chi phí – lợi ích được xây dựng, hệ thống tạo ra giá trị kinh tế ròng dương là 255.815. Điều này cho thấy cách tiếp cận của đề tài không dừng ở mức thử thuật toán, mà đã tiến gần tới tư duy của một giải pháp dữ liệu phục vụ quyết định quản trị.

5.2. Kiến nghị thực tế

Thứ nhất, với kết quả hiện tại, mô hình nên được sử dụng như một hệ thống cảnh báo và hỗ trợ rà soát thủ công thay vì cơ chế tự động chặn giao dịch. Lý do là Recall đã đủ cao để hỗ trợ phát hiện fraud, nhưng Precision còn thấp nên nếu tự động hóa mạnh sẽ ảnh hưởng tới nhiều giao dịch hợp lệ.

Thứ hai, doanh nghiệp hoặc tổ chức triển khai thực tế nên thay bộ giả định chi phí trong báo cáo bằng số liệu vận hành thật. Khi đó, net business value sẽ trở thành một KPI có thể theo dõi định kỳ, giúp điều chỉnh threshold phù hợp với chính sách rủi ro và năng lực xử lý cảnh báo của đơn vị.

Thứ ba, dashboard nên được mở rộng theo hướng hỗ trợ vòng lặp phản hồi nghiệp vụ. Khi một cảnh báo được xác nhận là fraud thật hoặc bị bác bỏ, trạng thái đó cần được ghi lại để cập nhật summary, đồng thời có thể làm dữ liệu phản hồi cho những lần tái huấn luyện sau.

Thứ tư, về hạ tầng, nếu quy mô dự đoán tăng lên trong thực tế, nên cân nhắc triển khai micro-batch thực sự với Spark Structured Streaming hoặc một nền tảng streaming tương đương để giảm độ trễ và tăng tính tự động cho speed path.

5.3. Hạn chế của đề tài

Hạn chế thứ nhất là dữ liệu của đề tài là dữ liệu công khai từ Kaggle, không phải dữ liệu phát sinh từ một tổ chức cụ thể. Vì vậy, các giả định chi phí – lợi ích và một số diễn giải nghiệp vụ mới dừng ở mức mô phỏng. Hạn chế thứ hai là bộ artifact hiện có chưa xuất ra bảng feature importance chuẩn hóa, nên phần business insight được suy luận từ EDA và logic đặc trưng thay vì từ một biểu đồ xếp hạng biến độc lập.

Hạn chế thứ ba là kiến trúc gần thời gian thực mới dừng ở mức mô phỏng học thuật trên Colab – MongoDB – Streamlit – ngrok. Dù logic hệ thống đã rõ ràng, đây chưa phải là một triển khai production-grade với cơ chế streaming, giám sát lỗi, logging, phân quyền và tối ưu hiệu năng đầy đủ. Hạn chế thứ tư là số lượng FP ở ngưỡng vận hành còn lớn, khiến mô hình phù hợp hơn với cảnh báo và rà soát thủ công thay vì tự động hóa mức cao.

5.4. Định hướng phát triển

Trong các nghiên cứu tiếp theo, có thể mở rộng đề tài theo bốn hướng chính. Thứ nhất, thử nghiệm các kỹ thuật mô hình hóa nâng cao hơn như LightGBM, CatBoost hoặc các mô hình học sâu chuyên biệt cho dữ liệu chuỗi giao dịch. Thứ hai, xây dựng thêm các đặc trưng hành vi phức tạp hơn như rolling statistics, velocity features hoặc graph-based features giữa khách hàng – thẻ – merchant. Thứ ba, chuyển phần đánh giá kinh tế từ mức giả định sang mức dựa trên dữ liệu vận hành thật, qua đó biến dashboard thành một công cụ giám sát đồng thời cả rủi ro và hiệu quả kinh doanh. Thứ tư, mở rộng kiến trúc theo hướng streaming thực tế với Spark/Kafka để tiến gần hơn tới một hệ thống dữ liệu lớn đầy đủ cho fraud detection.

TÀI LIỆU THAM KHẢO

- Apache Spark. (2025). *Structured Streaming Programming Guide*. Retrieved from <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- Caixabank Tech. (2024). *Transactions Fraud Datasets*. Retrieved from Kaggle: <https://www.kaggle.com/datasets/computingvictor/transactions-fraud-datasets>
- Garcia, H. H. (2009). *Learning from Imbalanced Data, IEEE Transactions on Knowledge and Data Engineering*.
- Guestrin, T. C. (2016). “*XGBoost: A Scalable Tree Boosting System*,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Guyen, A. L. (2016). *A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection*.
- MongoDB. (2025). *MongoDB Manual – Data Modeling and Indexing*. Retrieved from <https://www.mongodb.com/docs/manual/>
- N. V. Chawla, K. W. (2002). *SMOTE: Synthetic Minority Over-sampling Technique, Journal of Artificial Intelligence Research*.